

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Over the past two decades, industrial maintenance strategies have undergone significant evolution, transitioning from reactive fault-response approaches to predictive, data-driven methodologies. Traditional maintenance models - reactive maintenance that addresses failures only after they occur, and preventive maintenance that relies on fixed schedules - result in either catastrophic equipment failures causing costly downtime or unnecessary servicing leading to inflated maintenance expenditures. Industrial equipment failures cause average production losses exceeding 10–15% annually, with unexpected breakdowns accounting for 30–40% of total maintenance costs in manufacturing sectors.

The emergence of Industry 4.0 and the Internet of Things (IoT) has created opportunities for intelligent, real-time monitoring systems capable of predicting equipment failures before they occur. Modern industrial equipment generates continuous streams of sensor data - temperature fluctuations, vibration patterns, three-phase stator current variations, and shaft speed deviations - that contain valuable diagnostic information. However, traditional industrial communication protocols such as Classical CAN (Controller Area Network) are insufficient for multi-sensor, high-throughput data collection due to their speed limitation of 1 Mbps maximum and restricted payload capacity of 8 bytes per frame.

CAN FD (CAN with Flexible Data-rate) emerges as a transformative protocol, supporting data transmission rates up to 8 Mbps and payload capacities of up to 64 bytes per frame, enabling efficient multi-parameter sensor network architectures. When combined with edge computing platforms such as the Raspberry Pi 4 and unsupervised machine learning algorithms deployed at the network edge, real-time anomaly detection and predictive maintenance become feasible without the latency and data privacy concerns associated with cloud-only processing.

This project develops a Smart Industrial Monitoring System that integrates high-speed CAN FD networking, ESP32 WROOM-32 based sensor nodes, Isolation Forest edge machine learning analytics on a Raspberry Pi 4 gateway, and a LAN-accessible real-time web dashboard to enable truly predictive maintenance operations across multiple industrial machines simultaneously.

1.2 RELEVANCE

This project holds critical relevance in today's rapidly transforming industrial landscape.

Industry 4.0 Context: The integration of IoT, artificial intelligence, and advanced communication protocols represents the core of the fourth industrial revolution. Predictive maintenance stands as one of its flagship applications, enabling manufacturers to transition from reactive fault-response to proactive, intelligence-driven operational models. This project directly embodies these principles by combining industrial-grade communication with on-device machine learning.

Economic Impact: Unplanned downtime costs industrial companies significantly - manufacturing sectors lose an estimated \$50 billion annually due to equipment failures. Predictive maintenance has been demonstrated to reduce unplanned downtime by 30-40%, translating directly into cost savings and sustained productivity improvements. By detecting fault signatures early across multiple machines, this system enables timely intervention before failures escalate.

Technological Advancement: The convergence of CAN FD's robust, high-bandwidth communication capabilities with edge-deployed Isolation Forest anomaly detection creates possibilities for on-device

intelligence. This reduces cloud dependency entirely while maintaining the real-time responsiveness critical for safety-sensitive industrial operations.

Scalability and Modularity: The system architecture supports monitoring of multiple machines on a shared CAN FD bus, with each machine assigned dedicated CAN frame identifiers and an independent machine learning model. This allows incremental expansion from monitoring individual equipment items to entire production facilities without redesigning the core architecture.

Real-Time Decision Making: By processing all sensor data locally at the Raspberry Pi 4 edge gateway, the system achieves sub-second latency for fault detection and alert generation - essential for preventing catastrophic failures in safety-critical applications. Automated email notifications further ensure that responsible personnel are alerted immediately upon anomaly detection, regardless of their physical location.

1.3 PROJECT UNDERTAKEN

The "Smart Industrial Monitoring System with Predictive Maintenance Using CAN FD Network and Edge Machine Learning Analytics" project develops an integrated industrial IoT platform designed to address modern manufacturing challenges through real-time monitoring and predictive intelligence.

The system physically monitors two industrial machines simultaneously: Motor 1, a three-phase induction motor, and Motor 2, a centrifugal pump. Each machine is served by a dedicated ESP32 WROOM-32 sensor node interfaced with an MCP2518FD CAN FD controller, transmitting structured sensor frames over a shared CAN FD bus to a central Raspberry Pi 4 gateway. The gateway runs an independent Isolation Forest anomaly detection model per machine and exposes all processed data through a Flask REST API to a LAN-accessible Chart.js web dashboard.

Primary Objectives:

- To implement a high-speed CAN FD sensor network capable of collecting multi-parameter equipment data - including temperature, three-axis vibration, three-phase stator currents (Ia, Ib, Ic), shaft speed, and ambient conditions - from multiple industrial machines simultaneously.
- To design ESP32 WROOM-32 based sensor nodes integrated with MCP2518FD CAN FD controllers, transmitting structured 32-byte little-endian CAN FD frames at configurable intervals over a shared industrial bus.
- To develop and deploy Isolation Forest unsupervised anomaly detection models on a Raspberry Pi 4 edge gateway, capable of classifying machine health as NORMAL, WARNING, or CRITICAL in real time without dependency on cloud infrastructure.
- To simulate and validate four motor operating scenarios - normal operation, bearing fault, stator winding fault, and rotor bar fault - and use the resulting sensor data to evaluate system detection performance.
- To create a LAN-accessible real-time web dashboard that visualizes live sensor readings, machine health status, anomaly scores, and historical trends for both monitored machines through a Flask REST API backend.
- To implement automated email alerting via SMTP that notifies maintenance personnel immediately upon WARNING or CRITICAL anomaly classification.
- To validate system performance through comprehensive experimentation across all four fault scenarios, documenting detection accuracy, alert latency, and API response performance.

System Capabilities:

- Real-time acquisition of temperature, vibration, three-phase stator currents, shaft speed, and ambient sensor data from two machines via a shared CAN FD bus.
- Per-machine Isolation Forest anomaly detection at the edge, eliminating cloud latency for critical fault classification decisions.
- Three-level health classification - NORMAL, WARNING, and CRITICAL - based on Isolation Forest decision function scores.
- LAN-accessible Chart.js web dashboard with live sensor trend charts, machine health indicators, and an anomaly event log updated at one-second intervals.
- Automated SMTP email alerts triggered on WARNING and CRITICAL classifications, delivering fault details to designated maintenance personnel.
- Scalable CAN FD bus architecture supporting extension to additional sensor nodes without modification to the gateway software.

1.4 ORGANIZATION OF PROJECT REPORT

This final report is structured into seven chapters, each addressing specific aspects of the project development and validation.

Chapter 1 - Introduction establishes the technical and industrial context for the project, presents its objectives, describes system capabilities, and outlines the report structure.

Chapter 2 - Literature Survey reviews ten standard research papers covering CAN FD protocol advancements, machine learning algorithms for predictive maintenance, edge computing architectures, and real-time industrial monitoring systems. It analyzes methodologies, advantages, limitations, and identifies the research gaps this project addresses.

Chapter 3 - Design and Drawing presents the complete system architecture through hierarchical block diagrams, CAN FD network topology, hardware component specifications, and the software pipeline design. Component selection rationale and system integration approach are documented in detail.

Chapter 4 - Implementation details the hardware assembly, sensor-to-microcontroller wiring, ESP32 WROOM-32 firmware development, Raspberry Pi 4 gateway software configuration, Isolation Forest model training procedure, Flask REST API setup, and end-to-end integration testing.

Chapter 5 - Experimentation describes the experimental procedures and test cases conducted across all four operating scenarios - normal, bearing fault, stator fault, and rotor bar fault - documenting sensor behavior, anomaly detection response, API performance, and dashboard update functionality with supporting result tables.

Chapter 6 - Results and Discussion analyzes and interprets the experimental findings, evaluates system performance against project objectives, discusses technical insights and limitations encountered during deployment, and compares achieved results against relevant industry benchmarks.

Chapter 7 - Conclusion synthesizes the project's technical contributions, summarizes key achievements, acknowledges limitations, and outlines future enhancement directions including scalability, model generalization, and cloud integration.

References provides a comprehensive bibliography of 25 sources in IEEE format, including research papers, technical standards, component datasheets, and software documentation.

1.5 SUMMARY

This chapter has introduced the Smart Industrial Monitoring System with Predictive Maintenance project, establishing its technical foundation and industrial significance. The background section outlined the limitations of traditional maintenance strategies and conventional industrial communication protocols, and positioned CAN FD networking combined with edge machine learning as an effective solution for real-time predictive maintenance. The relevance section justified the project through Industry 4.0 alignment, quantified economic impact, technological advancement, and the practical value of real-time edge-based decision making. The project undertaken section detailed the system's physical realization - monitoring two industrial machines over a shared CAN FD bus, with per-machine Isolation Forest anomaly detection, a Flask REST API, a LAN-accessible web dashboard, and automated email alerting across four validated fault scenarios. Subsequent chapters present the complete design, implementation, experimental validation, and analytical findings that collectively demonstrate the system's effectiveness as a practical, deployment-ready industrial predictive maintenance platform.

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION

The foundation of this project rests on extensive research into industrial communication protocols, machine learning techniques for predictive maintenance, edge computing architectures, and real-time monitoring dashboards. This chapter reviews ten peer-reviewed research papers and technical documents that informed the system design and justified key architectural decisions. The literature survey encompasses four critical domains: (1) CAN FD protocol advantages and implementations, (2) machine learning algorithms for equipment failure prediction, (3) edge computing for real-time industrial decision-making, and (4) real-time dashboard design for operational visibility. Each paper was evaluated based on methodology rigor, practical applicability, reported performance metrics, and alignment with the objectives of this project.

2.2 LITERATURE SURVEY TABLE

Table 2.2.1: Literature Survey

Sr. No	Title of Paper	Methodology Used	Pros	Cons	Year
1	Security Architecture for Automotive Communication Networks: CAN FD Protocol Enhancement	HMAC/SHA-2 authentication with AES-256 encryption for CAN FD networks	Enhanced security, 8 Mbps data transmission, 64-byte payload capacity	Complex implementation, requires specialized security expertise	2023
2	Optimizing Predictive Maintenance with Machine Learning and IoT: A Business Strategy for Reducing Downtime	Supervised, unsupervised, and reinforcement learning algorithms with IoT sensor data	30-40% downtime reduction, 20-30% maintenance cost decrease	High implementation costs, specialized skill requirements	2024
3	Edge Computing for Real-Time Decision-Making in Industrial Automation Systems	Three-Real-Time architecture with edge computing and virtualization scheduling	Sub-millisecond response times, real-time decision making capability	Hardware complexity, potential reliability concerns	2025
4	Modelling and Performance Analysis of Time-Triggered CAN FD Protocol for Real-time Distributed Control Systems	Time-triggered CAN FD with periodic frame transmission and arbitration-free access	Higher transmission rates, reduced access delays, deterministic communication	Limited compatibility with existing priority-based systems	2024
5	AI-Powered Predictive Maintenance in IoT-Enabled Smart Factories	CNN and RNN algorithms integrated with IoT devices for failure prediction	Superior accuracy with deep learning, real-time failure prediction	Data privacy concerns, high computational requirements	2025

6	Three-Real-Time Architecture of Industrial Automation Based on Edge Computing	Edge computing model with real-time hardware network and operating system	AI integration capability, replacing traditional industrial controllers	Integration challenges with legacy systems	2019
7	Anomaly Detection in Smart Industrial Machinery Plant Using IoT and Machine Learning	CNN, SVM, and Random Forest algorithms with IoT sensors and cameras	13% improvement in anomaly detection, 3% decrease in false positives	CNN requires extensive training data, computational overhead	2023
8	Predictive Maintenance in Industrial IoT Using Machine Learning Approach	AdaBoost machine learning algorithm with vibration analysis techniques	Effective fault classification, reduced operational downtime	Limited to vibration-based fault detection	2024
9	Industrial Motor Operational Risk Detection using IoT and Isolation Forest	Isolation Forest unsupervised anomaly detection applied to IoT sensor data (vibration, temperature, current) with rolling statistics feature engineering and normalization	99.13% precision and recall, 99.12% F1-score; effective early fault detection without labeled training data	Evaluated against synthetic ground truth labels; results may vary on fully unseen real-world fault distributions	2025
10	Assessment of IoT-Driven Predictive Maintenance Systems for Industrial Equipment	IoT sensors with machine learning algorithms for equipment breakdown prediction	97.58% accuracy in breakdown prediction, practical industrial validation	Limited to specific equipment types, requires extensive setup	2024

2.3 KEY RESEARCH FINDINGS

CAN FD Protocol Capabilities (2023-2024):

- CAN FD supports data transmission rates up to 8 Mbps, representing an eightfold improvement over Classical CAN's 1 Mbps ceiling, enabling high-throughput multi-sensor data collection on a shared industrial bus.
- Payload capacity is increased to 64 bytes per frame compared to 8 bytes in Classical CAN, allowing all primary sensor parameters - temperature, vibration, three-phase stator currents, shaft speed, and fault identifier - to be packed into a single 32-byte CAN FD frame without frame splitting.
- Dynamic bit rate switching supports a lower arbitration phase rate (500 Kbps) for fair bus access among multiple nodes, while the data phase operates at up to 2 Mbps for efficient payload delivery - a configuration adopted directly in this project.
- Enhanced security models incorporating HMAC/SHA-2 authentication and AES-256 encryption have been demonstrated to address cybersecurity vulnerabilities in CAN FD deployments.
- Deterministic communication guarantees provided by CAN FD meet the real-time requirements of safety-critical industrial monitoring applications.

Machine Learning for Predictive Maintenance (2024-2025):

- Supervised algorithms such as Random Forest achieve 85–95% accuracy in equipment failure prediction, while CNN-based approaches have been reported to reach up to 97.58% accuracy in equipment breakdown classification.
- LSTM networks demonstrate strong capability in time-series analysis, effectively capturing temporal degradation patterns in sequential sensor data.
- Hybrid approaches combining multiple supervised algorithms offer superior robustness but require extensive labeled fault datasets for training, which are rarely available in real industrial deployments.
- Unsupervised methods, including Isolation Forest, have been demonstrated to be effective for anomaly detection without requiring labeled failure data, making them particularly well-suited for industrial environments where normal operating data is abundant but labeled fault examples are scarce.
- This project adopts the Isolation Forest algorithm precisely for this reason - it learns the normal operating envelope of each machine from unlabeled sensor data and flags deviations as anomalies, eliminating the labeled-data dependency that constrains supervised approaches.
- Sivanantham et al. demonstrated that Isolation Forest applied to high-frequency IoT sensor data - vibration, temperature, and current - from industrial motors achieves 99.13% precision and recall with a 99.12% F1-score, using rolling statistics and normalization as feature engineering steps. This directly validates the algorithm selection in this project and confirms that Isolation Forest is capable of detecting early fault signatures in motor sensor data without requiring labeled fault examples for training.

Edge Computing Performance (2019-2025):

- Sub-millisecond inference response times are achievable on Raspberry Pi 4 class hardware with optimized machine learning models, confirming its suitability as an industrial edge computing platform.
- Local edge processing eliminates cloud round-trip latency, reducing fault detection delays to tens of milliseconds and enhancing data privacy by keeping sensitive operational data within the facility network.
- The Three-Real-Time architecture - combining real-time hardware networks, real-time operating systems, and scheduling virtualization - demonstrates that AI inference can be reliably deployed on resource-constrained edge devices in industrial automation contexts.
- Edge-deployed Flask REST APIs have been validated as effective mechanisms for exposing processed sensor data and machine health status to LAN-accessible dashboards with consistently low response latency.

Real-Time Dashboard Design (2023-2024):

- Effective industrial dashboards combine real-time sensor metrics updated at sub-second intervals, historical trend visualization, and predictive health indicators to support informed operational decision-making.
- Multi-channel alerting mechanisms - including email notifications triggered by anomaly classification events - significantly improve maintenance response efficiency by ensuring personnel are notified regardless of their physical proximity to the dashboard.

- Chart.js based web frontends polling REST API backends at one-second intervals have been validated as a practical and lightweight approach to real-time industrial data visualization without requiring dedicated SCADA infrastructure.

2.4 RESEARCH GAPS ADDRESSED BY THIS PROJECT

A review of the existing literature identified the following gaps that this project directly addresses:

Gap 1 - Limited Integration of CAN FD with Edge Machine Learning: The majority of reviewed research addresses CAN FD communication and machine learning for predictive maintenance as separate domains. Few published frameworks demonstrate a unified architecture where high-speed CAN FD sensor networking is directly coupled with on-device machine learning inference at the edge gateway. This project bridges that gap by integrating both components into a single, physically deployed system.

Gap 2 - Absence of Unsupervised Fault Detection on CAN FD Networks: Existing CAN FD-based monitoring implementations predominantly rely on threshold-based or rule-based fault detection. The application of unsupervised anomaly detection - specifically Isolation Forest - directly on CAN FD frame data streams, with per-machine independent models and three-level severity classification, represents a contribution not addressed in the reviewed literature.

Gap 3 - Multi-Machine Monitoring on a Shared CAN FD Bus: Reviewed studies typically demonstrate single-machine monitoring scenarios. This project validates simultaneous monitoring of two distinct industrial machines - a three-phase induction motor and a centrifugal pump - on a shared CAN FD bus, with independent anomaly detection models and separate frame identifier spaces (0x100/0x101 for Motor 1, 0x200/0x201 for Motor 2).

Gap 4 - Practical Fault Scenario Validation: A gap exists between academic algorithm evaluations and physically grounded fault scenario testing. This project addresses this through physics-based simulation of four validated motor fault scenarios - normal operation, bearing fault, stator winding fault, and rotor bar fault - using a purpose-built sensor simulator that models real induction motor fault signatures including bearing pass frequencies, stator current harmonics, and rotor bar sideband components.

Gap 5 - End-to-End System Latency Characterization: Few studies characterize end-to-end latency from sensor data acquisition through CAN FD transmission, edge ML inference, REST API serving, and dashboard rendering as an integrated pipeline. This project measures and reports latency at each stage, providing a practical reference for industrial IoT system designers.

Project Contributions:

- Demonstrates a unified, physically deployed architecture integrating CAN FD sensor networking, Raspberry Pi 4 edge computing, and per-machine Isolation Forest anomaly detection.
- Validates simultaneous multi-machine monitoring on a shared CAN FD bus with independent machine learning models per machine.
- Evaluates system performance across four physics-based motor fault scenarios, documenting sensor signatures, anomaly scores, and detection response for each fault type.
- Establishes a practical end-to-end latency profile from CAN FD frame reception through ML inference, API serving, and dashboard update.
- Implements automated SMTP email alerting integrated with the anomaly classification pipeline, enabling remote fault notification without cloud infrastructure dependency.

2.5 SUMMARY

This chapter reviewed ten peer-reviewed research papers and technical documents spanning CAN FD protocol advances, machine learning algorithms for predictive maintenance, edge computing architectures, and real-time industrial monitoring systems. Key findings established that CAN FD enables an eightfold throughput improvement over Classical CAN with deterministic communication guarantees, supervised machine learning algorithms achieve 85–97% accuracy in fault classification when labeled training data is available, and edge computing platforms such as the Raspberry Pi 4 are capable of supporting real-time ML inference with sub-millisecond response times. The survey further identified that unsupervised methods such as Isolation Forest are particularly appropriate for industrial deployments where labeled fault data is unavailable - a finding that directly motivated the algorithm selection in this project. Critical gaps were identified in the integration of CAN FD with edge machine learning, multi-machine monitoring on shared buses, unsupervised fault detection on CAN FD networks, and end-to-end system latency characterization - all of which this project directly addresses. Subsequent chapters present the complete system design, hardware and software implementation, experimental validation across all four fault scenarios, and a comprehensive analysis of the results.

CHAPTER 3
DESIGN AND DRAWING

3.1 INTRODUCTION

The Smart Industrial Monitoring System is designed around a layered hardware-software architecture that integrates distributed multi-parameter sensor acquisition, high-speed industrial CAN FD communication, edge-based machine learning, and real-time web visualization. The physical system comprises two ESP32 WROOM-32 sensor nodes - one monitoring a three-phase induction motor and one monitoring a centrifugal pump - each interfaced with dedicated sensors for temperature, three-axis vibration, three-phase stator current, shaft speed or flow rate, and ambient conditions. Both nodes communicate over a shared CAN FD bus to a central Raspberry Pi 4 gateway, which runs independent per-machine Isolation Forest anomaly detection models, generates automated email alerts, and serves all processed data through a Flask REST API to a LAN-accessible Chart.js web dashboard.

This chapter presents the complete system design through a hierarchical block diagram, hardware component specifications, CAN FD network topology including frame identifier assignments, and the edge software pipeline architecture. The design prioritizes real-time responsiveness, industrial communication reliability, edge processing efficiency, and deployment feasibility within a small-scale manufacturing environment.

3.2 SYSTEM OVERVIEW

The system is organized as a three-tier architecture, with each tier performing distinct and complementary functions.

Tier 1 - Sensor Nodes (ESP32 WROOM-32): Two independent sensor nodes are deployed, one per monitored machine. Each node is built around an ESP32 WROOM-32 microcontroller interfaced with an MCP2518FD CAN FD controller via SPI. The first node monitors Motor 1, a three-phase induction motor, and is interfaced with a DS18B20 surface temperature sensor, an MPU-6050 three-axis accelerometer for vibration measurement along the X, Y, and Z axes, an ACS712 Hall-effect current sensor for three-phase stator current measurement (I_a, I_b, I_c), and an incremental encoder for shaft speed measurement in RPM. The second node monitors Motor 2, a centrifugal pump, and carries an identical sensor configuration with a flow meter replacing the shaft speed encoder for volumetric flow rate measurement. Both nodes additionally interface with a DHT11 sensor for ambient temperature and relative humidity monitoring. Each node samples all sensor channels, assembles the readings into structured CAN FD frames, and transmits them over the shared physical bus at configured intervals.

Tier 2 - Edge Gateway (Raspberry Pi 4): The Raspberry Pi 4 Model B serves as the central edge processing unit. It receives all CAN FD frames from both sensor nodes via a SocketCAN interface, parses and validates incoming data, and routes each frame to the appropriate machine-specific processing pipeline. Each pipeline maintains a rolling data buffer, extracts statistical features, and passes the feature vector to a dedicated per-machine Isolation Forest model for unsupervised anomaly detection. The model classifies each reading as NORMAL, WARNING, or CRITICAL based on the Isolation Forest decision function score. On WARNING or CRITICAL classification, the system generates a timestamped alert record and dispatches an automated SMTP email notification to designated maintenance personnel. All processed data - live sensor readings, health classifications, anomaly scores, alert logs, and email trigger functionality - are exposed through a Flask REST API.

Tier 3 - Dashboard and Visualization: A Chart.js-based web dashboard, accessible from any device on the local network via browser, retrieves live sensor data and machine health status from the Flask REST API at one-second polling intervals. The dashboard renders real-time trend charts for all sensor channels of both

machines, displays current health classification indicators, maintains a live anomaly event log, and provides an interface for triggering email alert reports on demand.

3.3 BLOCK DIAGRAM

Smart Industrial Monitoring System Architecture

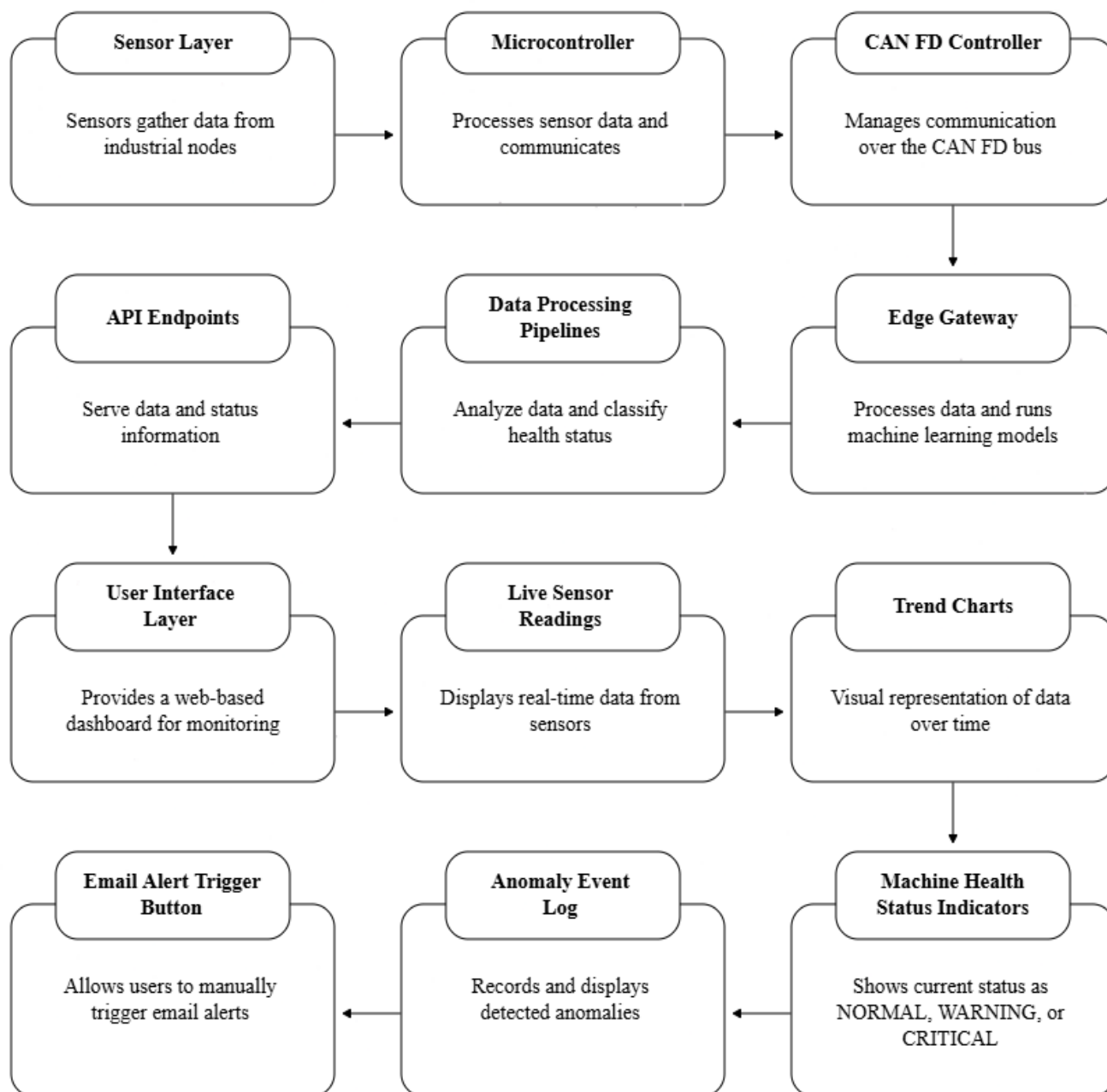


Figure 3.3.1: Smart Industrial Monitoring System Block Diagram

3.4 BLOCK DIAGRAM DESCRIPTION

Sensor Acquisition Layer:

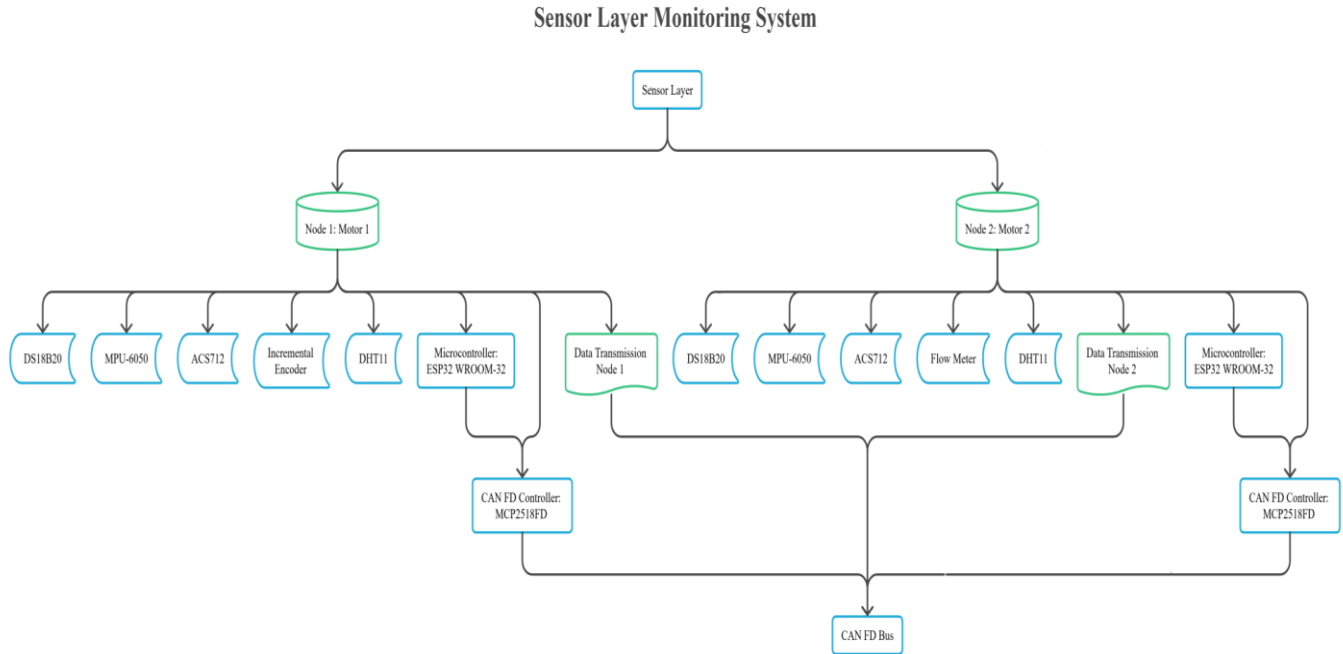


Figure 3.4.1: Sensor Layer Detail Diagram

The sensor acquisition layer consists of two parallel ESP32 WROOM-32 based sensor nodes, each dedicated to one monitored machine. The following sensor channels are acquired by each node:

- The DS18B20 digital temperature sensor communicates via the 1-Wire protocol and provides calibrated equipment surface temperature readings with $\pm 0.5^{\circ}\text{C}$ accuracy, suitable for detecting thermal anomalies associated with bearing friction, stator winding faults, and rotor losses.
- The MPU-6050 six-axis IMU is used as a three-axis accelerometer to measure radial vibration along the X and Y axes and axial vibration along the Z axis in units of g. Vibration signatures captured across all three axes enable detection of mechanical faults including bearing defects, rotor imbalance, and misalignment.
- The ACS712 Hall-effect current sensor measures three-phase stator currents I_a , I_b , and I_c in amperes. Deviations in current balance, harmonic content, and sideband components in the current spectrum are primary indicators of stator winding faults and broken rotor bar conditions.
- An incremental encoder (Motor 1) measures shaft rotational speed in RPM. Speed fluctuations and drops from rated speed are indicative of rotor bar faults and increased mechanical loading.
- A flow meter (Motor 2) measures volumetric flow rate, serving as the equivalent operational parameter for the centrifugal pump in place of shaft speed.
- The DHT11 sensor provides ambient temperature and relative humidity readings over a single-wire interface, transmitted in a separate secondary CAN FD frame to supplement equipment-level condition monitoring with environmental context.

Each ESP32 WROOM-32 node samples all sensor channels, assembles the readings into structured CAN FD frames, and transmits them over the shared physical bus to the Raspberry Pi 4 gateway.

Communication Network:

- The CAN FD bus physically connects both ESP32 sensor nodes and the Raspberry Pi 4 gateway using shielded twisted-pair cable terminated with 120 Ω resistors at both ends of the bus to ensure impedance matching and signal integrity.
- CAN FD is configured with a 500 Kbps arbitration phase bit rate and a 2 Mbps data phase bit rate, utilizing standard 11-bit frame identifiers and data payloads of up to 64 bytes per frame.
- Motor 1 transmits a 32-byte primary frame under CAN ID 0x100, carrying node identifier, temperature, vibration (X, Y, Z), stator currents (Ia, Ib, Ic), shaft speed, and fault identifier. Its secondary 8-byte ambient frame is transmitted under CAN ID 0x101.
- Motor 2 transmits an equivalent primary frame under CAN ID 0x200 and a secondary ambient frame under CAN ID 0x201, allowing the gateway to unambiguously route incoming frames to the correct machine processing pipeline by frame identifier alone.
- The protocol's built-in error detection mechanisms - cyclic redundancy check (CRC), bit monitoring, and acknowledgement verification - ensure reliable data delivery under electrically noisy industrial conditions per ISO 11898-1.

Edge Processing Gateway:

- The Raspberry Pi 4 receives all CAN FD frames via the SocketCAN interface and routes each frame to its machine-specific processing pipeline based on the received frame identifier.
- Parsed sensor values are stored in a per-machine rolling time-window buffer, from which statistical features are extracted and passed to the corresponding Isolation Forest model.
- Each machine is served by an independent Isolation Forest model trained exclusively on that machine's normal operating data, ensuring that the anomaly detection baseline is specific to each machine's individual operating characteristics.
- The Isolation Forest decision function score determines the health classification for each inference cycle: a score above 0 is classified as NORMAL; a score between -0.10 and 0 is classified as WARNING; a score below -0.10 is classified as CRITICAL.
- On WARNING or CRITICAL classification, the system generates a timestamped alert record and dispatches an automated SMTP email notification containing the machine identifier, classification level, anomaly score, and the associated sensor readings to designated maintenance personnel.
- Periodic sensor log reports summarizing machine health history are also available for email dispatch via the `/api/email_alert` endpoint, enabling scheduled reporting without manual dashboard access.
- All processed data is simultaneously exposed through the Flask REST API for dashboard consumption and external integration.

User Interface:

- The Chart.js web dashboard polls the Flask REST API at one-second intervals to retrieve current sensor values, health classifications, anomaly scores, and alert records for both machines.
- Separate real-time trend charts are rendered for temperature, vibration (X, Y, Z), stator currents (Ia, Ib, Ic), and speed or flow rate for each monitored machine.

- Health classification indicators display the current NORMAL, WARNING, or CRITICAL status for each machine prominently on the dashboard.
- A live anomaly event log records all WARNING and CRITICAL events with timestamps, machine identifiers, and associated sensor readings for operator review.
- The dashboard is accessible from any device on the local network, supporting continuous equipment surveillance without requiring physical presence on the production floor.

3.5 HARDWARE COMPONENTS SPECIFICATION

Table 3.5.1: Hardware Component List

Sr. No.	Component	Specification	Qty.	Justification
1	ESP32 WROOM-32	Dual-core Xtensa LX6, 240 MHz, 520 KB SRAM, Wi-Fi/BLE, 3.3V operating voltage	2	One node per monitored machine; sufficient processing capability for multi-channel sensor acquisition, frame assembly, and CAN FD transmission
2	MCP2518FD CAN FD Controller	Up to 8 Mbps CAN FD, SPI interface, -40°C to $+85^{\circ}\text{C}$ operating range	2	Provides full CAN FD protocol support for the ESP32; proven reliability in automotive and industrial communication applications
3	MCP2542 CAN Transceiver	Differential CAN bus signalling, industrial EMI immunity, 3.3V/5V compatible	2	Converts logic-level SPI signals to differential CAN bus voltage levels; provides robust immunity to electromagnetic interference
4	DS18B20 Temperature Sensor	$\pm 0.5^{\circ}\text{C}$ accuracy, -55°C to $+125^{\circ}\text{C}$ range, 1-Wire interface	2	Digital output eliminates ADC-related noise; suitable for direct equipment surface temperature monitoring
5	MPU-6050 IMU	Three-axis accelerometer $\pm 16g$, three-axis gyroscope, I2C interface, 16-bit ADC	2	Provides calibrated three-axis vibration measurements (X, Y, Z) for detection of bearing faults, rotor imbalance, and mechanical misalignment
6	ACS712 Current Sensor	30A range, Hall-effect sensing, 66 mV/A sensitivity, 5V supply	6	Three sensors per machine (one per phase) for non-invasive three-phase stator current measurement; enables detection of stator winding and rotor bar faults
7	Incremental Encoder	600 PPR, 5V supply, A/B quadrature output	1	Measures Motor 1 shaft speed in RPM; speed deviations indicate rotor bar faults and abnormal mechanical loading
8	Flow Meter	Pulse output, 1–30 L/min range, 5V supply	1	Measures Motor 2 (centrifugal pump) volumetric flow rate as the equivalent operational parameter to shaft speed

9	DHT11 Humidity and Temperature Sensor	$\pm 2^{\circ}\text{C}$ temperature accuracy, $\pm 5\%$ RH accuracy, single-wire interface	2	Provides ambient environmental data transmitted in secondary CAN FD frames for comprehensive condition monitoring context
10	Raspberry Pi 4 Model B	Quad-core ARM Cortex-A72, 1.5 GHz, 4 GB LPDDR4 RAM, Gigabit Ethernet	1	Provides sufficient computational capacity for dual-machine Python ML inference, Flask API hosting, and SocketCAN data management
11	Shielded Twisted-Pair Cable	24 AWG, 120 Ω characteristic impedance, shielded	As required	Maintains CAN FD signal integrity over the deployed bus length; shielding suppresses industrial EMI
12	Termination Resistors	120 Ω , $\frac{1}{4}$ W, metal film	2	Required at both ends of the CAN FD bus for impedance matching; prevents signal reflection and communication errors
13	DC Power Supply	5V regulated (Raspberry Pi); 3.3V regulated (ESP32 and sensors)	1	Stable regulated supply is essential for accurate sensor readings and reliable microcontroller operation

3.6 CAN FD NETWORK TOPOLOGY

The system employs a linear bus topology, as recommended for CAN FD deployments, with both ESP32 WROOM-32 sensor nodes and the Raspberry Pi 4 gateway connected along a shared shielded twisted-pair cable. A 120 Ω termination resistor is placed at each physical endpoint of the bus to match the cable's characteristic impedance and prevent signal reflections that could corrupt CAN FD frames.

The network is configured as follows:

- Arbitration phase bit rate: 500 Kbps
- Data phase bit rate: Up to 2 Mbps
- Frame format: CAN FD extended frame (11-bit identifier, up to 64 data bytes)
- Error handling: Built-in CRC error detection, acknowledgement checking, and automatic error frame signaling per ISO 11898-1 standard
- Bus termination: 120 Ω resistors at both physical endpoints of the bus

CAN Frame Identifier Assignments:

Table 3.6.1: CAN Frame Identifier

Frame ID	Machine	Frame Type	Payload	Size
0x100	Motor 1 - Induction Motor	Primary	Node ID, temperature, vibration X/Y/Z, Ia/Ib/Ic, shaft speed, fault ID	32 bytes
0x101	Motor 1 - Induction Motor	Secondary	Ambient temperature, relative humidity	8 bytes
0x200	Motor 2 - Centrifugal Pump	Primary	Node ID, temperature, vibration X/Y/Z, Ia/Ib/Ic, shaft speed, fault ID	32 bytes
0x201	Motor 2 - Centrifugal Pump	Secondary	Ambient temperature, relative humidity	8 bytes

The gateway routes all incoming CAN FD frames exclusively by frame identifier, requiring no additional addressing overhead. This topology ensures deterministic, low-latency data delivery from both sensor nodes simultaneously, with inherent fault detection mechanisms maintaining communication reliability under industrial electromagnetic interference conditions.

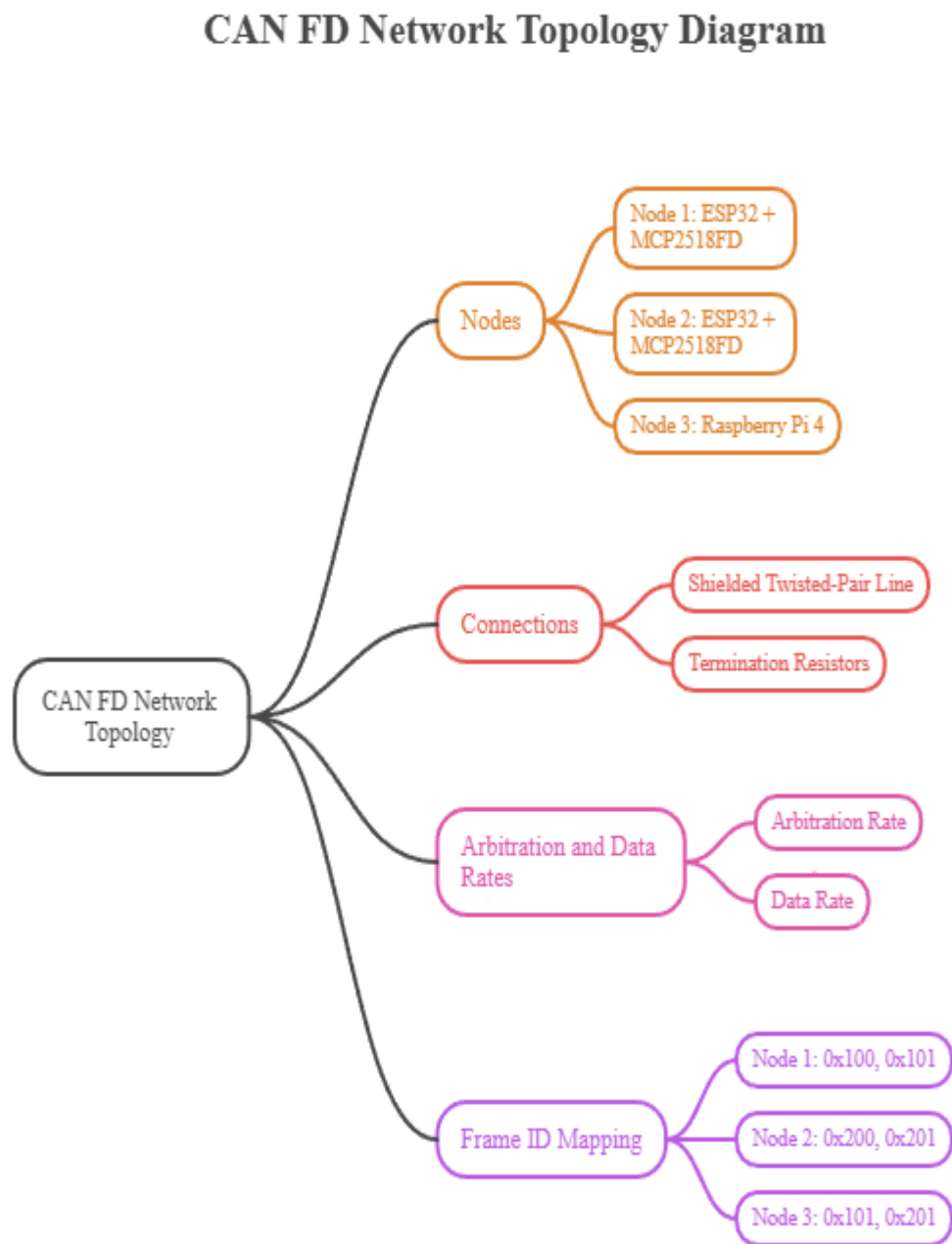
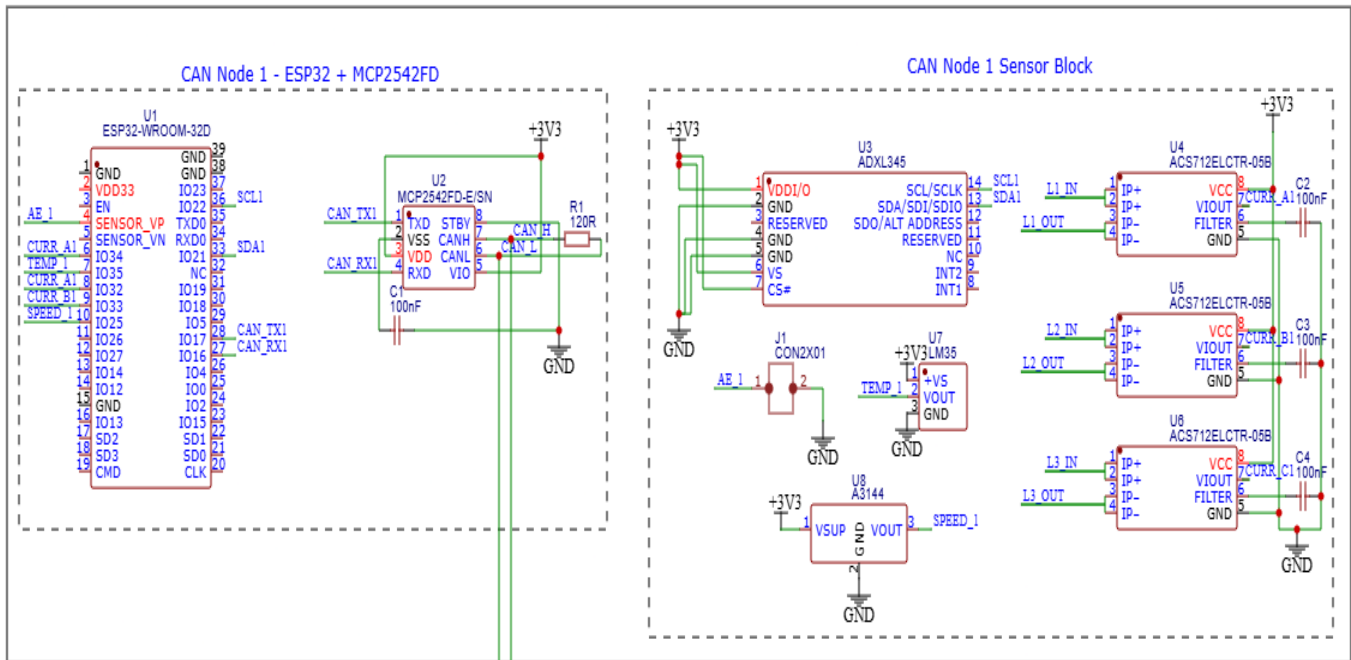


Figure 3.6.1: CAN FD Network Topology Diagram

3.6.1 Circuit Schematic

Motor 1 - Three Phase Induction Motor



Motor 2 - Centrifugal Pump

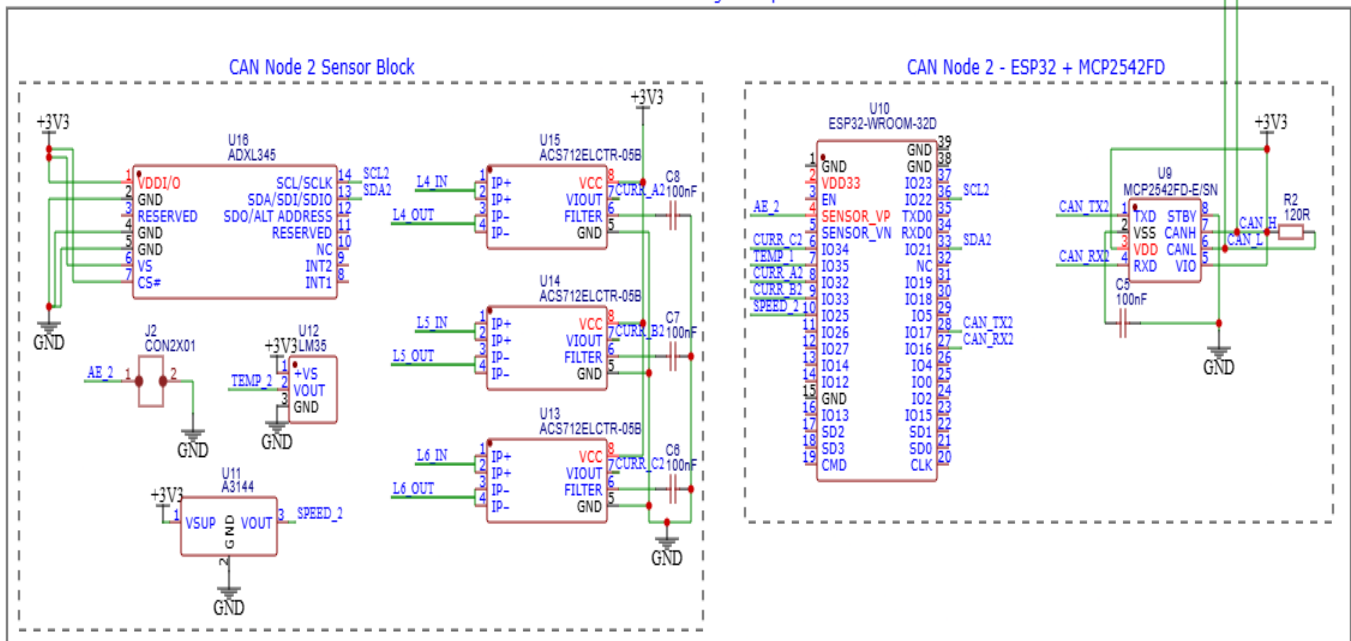


Figure 3.6.2: ESP32 Sensor Node Circuit Schematic

Figure 3.6.2 presents the complete circuit schematic of the ESP32 WROOM-32 sensor node, showing pin-level connections between the microcontroller, MCP2518FD CAN FD controller, MCP2542 transceiver, DS18B20, MPU-6050, ACS712 current sensors, DHT11, and incremental encoder. Both sensor nodes (Node 1 - Motor 1 and Node 2 - Motor 2) follow this identical schematic, with Node 2 substituting the incremental encoder with a flow meter on GPIO 36.

3.7 SOFTWARE ARCHITECTURE

Edge Processing Pipeline - Raspberry Pi 4:

1. CAN FD Frame Reception - The Python SocketCAN interface continuously receives incoming CAN FD frames from the physical bus and identifies each frame by its 11-bit standard identifier.
2. Frame Routing - Frames with IDs 0x100 and 0x101 are routed to the Motor 1 processing pipeline; frames with IDs 0x200 and 0x201 are routed to the Motor 2 processing pipeline. This routing is performed in real time without queuing delay.
3. Frame Parsing and Validation - Primary frames (0x100, 0x200) are unpacked from a 32-byte little-endian struct into named sensor fields: node ID, temperature, vibration X, vibration Y, vibration Z, Ia, Ib, Ic, speed or flow rate, and fault ID. Secondary frames (0x101, 0x201) are unpacked from an 8-byte struct into ambient temperature and relative humidity fields. CRC validity is confirmed by the SocketCAN interface before frames are passed to the pipeline.
4. Temporal Buffering - Parsed sensor readings for each machine are stored in a dedicated rolling time-window buffer of 200 samples, maintaining a continuously updated window of recent operating data for feature extraction.
5. Feature Extraction - Statistical features including mean, standard deviation, minimum, maximum, and rate of change are computed from the buffered sensor data for each channel, forming the feature vector passed to the Isolation Forest model.
6. ML Inference - Per-Machine Isolation Forest - Each machine is served by an independent Isolation Forest model trained on 200 samples of that machine's normal operating data collected during system initialization. The model computes a decision function score for each feature vector. Scores above 0 are classified as NORMAL; scores between -0.10 and 0 as WARNING; scores below -0.10 as CRITICAL.
7. Alert Generation - All classifications are logged with timestamps, machine identifiers, anomaly scores, and associated sensor readings. WARNING and CRITICAL classifications additionally trigger automated SMTP email notifications to designated maintenance personnel, delivering the machine identifier, fault severity, anomaly score, and sensor readings at the time of detection.
8. Log Report Dispatch - Periodic sensor health log reports summarizing recent machine operating history and anomaly events can be dispatched via email through the `/api/email_alert` endpoint, enabling scheduled reporting without requiring direct dashboard access.
9. Data Serving - Flask REST API - The following API endpoints are exposed by the Flask backend running on the Raspberry Pi 4:

Edge Software Pipeline Cycle

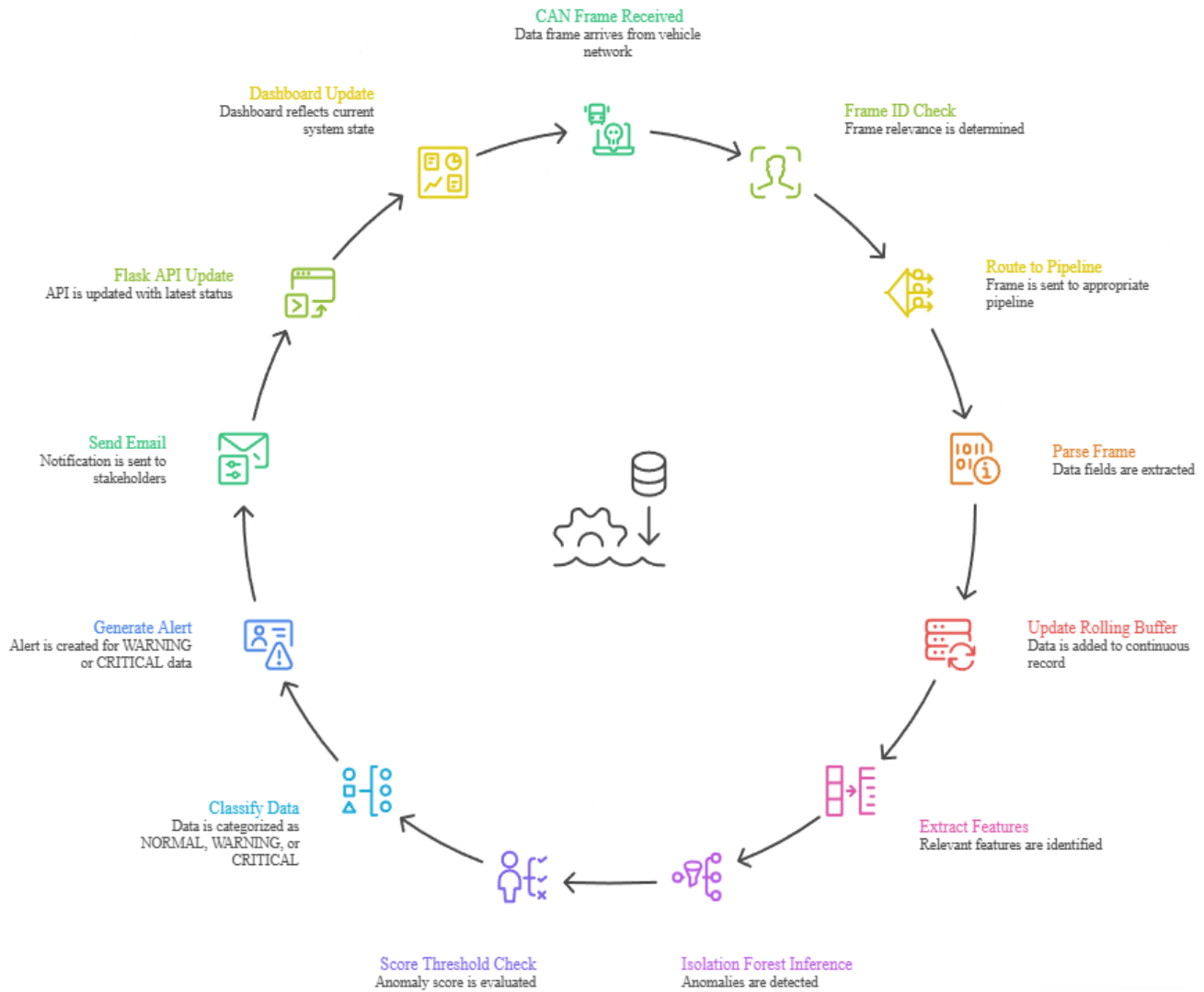


Figure 3.7.1: Edge Software Pipeline Flowchart

Table 3.7.1: Flask API's

Endpoint	Method	Description
/api/status	GET	Returns overall system health status and gateway uptime
/api/motor/1	GET	Returns live sensor readings and health classification for Motor 1
/api/motor/2	GET	Returns live sensor readings and health classification for Motor 2
/api/history	GET	Returns historical sensor readings and anomaly event log for both machines
/api/alerts	GET	Returns list of all WARNING and CRITICAL alert records with timestamps

<code>/api/email_alert</code>	POST	Triggers dispatch of a sensor log report email to designated recipients
-------------------------------	------	---

Dashboard Layer - Flask and Chart.js:

- The Chart.js frontend polls `/api/motor/1` and `/api/motor/2` at one-second intervals to retrieve and render live sensor readings and health classifications for both machines.
- `/api/history` is queried at configurable intervals to update historical trend charts and the anomaly event log displayed on the dashboard.
- `/api/alerts` populates the live alert feed, displaying all WARNING and CRITICAL events with machine identifiers, timestamps, and anomaly scores.
- Health classification indicators for each machine update dynamically, displaying NORMAL in green, WARNING in amber, and CRITICAL in red to provide immediate visual fault identification.
- The dashboard is accessible from any device on the local area network via browser, requiring no dedicated client software installation.

3.8 SUMMARY

This chapter has presented the complete system design for the Smart Industrial Monitoring System. The three-tier architecture - comprising two ESP32 WROOM-32 sensor nodes monitoring a three-phase induction motor and a centrifugal pump respectively, a shared CAN FD communication bus with dedicated frame identifier assignments per machine, a Raspberry Pi 4 edge gateway running independent per-machine Isolation Forest anomaly detection models, and a LAN-accessible Chart.js web dashboard - provides a fully integrated platform for real-time multi-machine equipment monitoring and predictive maintenance. Hardware components were selected on the basis of industrial suitability, sensor accuracy, communication compatibility, and deployment practicality. The CAN FD network topology with assigned frame identifiers (0x100/0x101 for Motor 1, 0x200/0x201 for Motor 2) ensures deterministic, unambiguous frame routing with built-in error detection. The edge software pipeline - from SocketCAN frame reception through per-machine Isolation Forest inference, three-level health classification, automated email alerting, and Flask REST API serving - enables low-latency anomaly detection and notification without dependency on cloud infrastructure. The design documented in this chapter forms the technical foundation for the implementation described in the following chapter.

CHAPTER 4

IMPLEMENTATION

4.1 INTRODUCTION

This chapter describes the physical realization of the Smart Industrial Monitoring System, detailing the hardware assembly, sensor integration, embedded firmware development, gateway software configuration, and system integration testing procedures carried out during the project. Implementation was executed in a systematic sequence of phases: component verification and preparation, sensor node assembly with pin-level wiring for both monitored machines, ESP32 WROOM-32 firmware development, Raspberry Pi 4 gateway configuration, per-machine Isolation Forest model deployment, Flask REST API and dashboard setup, and end-to-end integration testing.

The system as physically deployed comprises two ESP32 WROOM-32 sensor nodes - Node 1 monitoring Motor 1, a three-phase induction motor, and Node 2 monitoring Motor 2, a centrifugal pump. Each node is interfaced with a DS18B20 surface temperature sensor, an MPU-6050 three-axis accelerometer, three ACS712 Hall-effect current sensors for three-phase stator current measurement, a DHT11 ambient sensor, and either an incremental encoder (Motor 1) or a flow meter (Motor 2). Both nodes communicate over a shared CAN FD bus to a Raspberry Pi 4 gateway running independent per-machine Isolation Forest anomaly detection pipelines, a Flask REST API, and an SMTP email alerting system, with live data visualized on a LAN-accessible Chart.js web dashboard.

4.2 HARDWARE IMPLEMENTATION

Phase 1: Component Verification

Prior to assembly, all hardware components were individually verified against manufacturer datasheets. Supply voltage levels for the ESP32 WROOM-32 (3.3 V) and Raspberry Pi 4 (5 V) were confirmed using a calibrated multimeter. The MCP2518FD CAN FD controller was verified for SPI communication responsiveness on both nodes. The DS18B20 temperature sensor on each node was tested at ambient room temperature to confirm valid 1-Wire digital output. The MPU-6050 accelerometer was powered and queried over I2C to confirm valid three-axis acceleration readings. Each ACS712 current sensor was verified to produce the expected output voltage at zero current (2.5 V reference). The DHT11 sensor was powered and queried to confirm valid humidity and temperature readings within its specified accuracy range. The incremental encoder was verified for correct A/B quadrature pulse output under manual shaft rotation. The flow meter was verified to produce pulse output under controlled fluid flow.

Phase 2: Sensor Node Assembly - Motor 1 (Three-Phase Induction Motor)

All wiring connections for Node 1 were made on a breadboard for prototype deployment, with connections secured and verified against the pin mapping specified below. Shielded twisted-pair cable was used for the CAN FD bus connection. Wiring was carried out per the circuit schematic presented in Figure 3.6.2.

DS18B20 Temperature Sensor (1-Wire Protocol):

Table 4.2.1: DS18B20 Pin Connections

DS18B20 Pin	Connection
Pin 1 (GND)	ESP32 GND
Pin 2 (Data)	ESP32 GPIO 4 (4.7 k Ω pull-up resistor to 3.3 V)
Pin 3 (VCC)	ESP32 3.3 V

MPU-6050 Three-Axis Accelerometer (I2C Protocol):

Table 4.2.2: MPU-6050 Pin Connections

MPU-6050 Pin	Connection
VCC	ESP32 3.3 V
GND	ESP32 GND
SCL	ESP32 GPIO 22
SDA	ESP32 GPIO 21
INT	ESP32 GPIO 34

ACS712 Current Sensors - Phase A, Phase B, Phase C (Analog Output):

Table 4.2.3: ACS712 Pin Connections

ACS712 Pin	Connection
VCC	ESP32 V (via level shifter)
GND	ESP32 GND
OUT (Phase A)	ESP32 GPIO 35 (ADC1 Channel 7)
OUT (Phase B)	ESP32 GPIO 32 (ADC1 Channel 4)
OUT (Phase C)	ESP32 GPIO 33 (ADC1 Channel 5)

Incremental Encoder - Shaft Speed (Pulse Input):

Table 4.2.4: Encoder Pin Connections

Encoder Pin	Connection
VCC	ESP32 5 V
GND	ESP32 GND
Channel A	ESP32 GPIO 36
Channel B	ESP32 GPIO 39

DHT11 Ambient Sensor (Single-Wire Interface):

Table 4.2.5: DHT11 Pin Connections

DHT11 Pin	Connection
VCC	ESP32 3.3 V
GND	ESP32 GND
Data	ESP32 GPIO 27 (10 kΩ pull-up resistor to 3.3 V)

MCP2518FD CAN FD Controller (SPI Protocol):

Table 4.2.6: MCP2518FD Pin Connections

MCP2518FD Pin	Connection
---------------	------------

SCK	ESP32 GPIO 18
MOSI	ESP32 GPIO 23
MISO	ESP32 GPIO 19
CS	ESP32 GPIO 5
INT	ESP32 GPIO 26
VCC	ESP32 3.3 V
GND	ESP32 GND
CANH / CANL	Shielded twisted-pair to CAN FD bus

Phase 3: Sensor Node Assembly - Motor 2 (Centrifugal Pump)

Node 2 follows an identical wiring configuration to Node 1 with the following difference: the incremental encoder is replaced by a flow meter connected to ESP32 GPIO 36, which produces pulse output proportional to volumetric flow rate. All other sensor connections - DS18B20, MPU-6050, ACS712 (three phases), DHT11, and MCP2518FD - are wired to the same GPIO assignments as Node 1. Node 2 transmits under CAN frame IDs 0x200 (primary) and 0x201 (secondary), distinguishing its frames from Node 1's 0x100 and 0x101 transmissions on the shared bus.

A 120 Ω termination resistor was placed at each physical endpoint of the CAN FD bus - at Node 1 and at the Raspberry Pi 4 gateway - to match the cable's characteristic impedance and prevent signal reflection.

Phase 4: Raspberry Pi 4 Gateway Setup

The Raspberry Pi 4 was configured with Raspberry Pi OS (64-bit, Debian Bullseye). A MCP2518FD-based CAN FD interface module compatible with the Raspberry Pi SPI bus was installed and connected to the CANH and CANL lines of the shared bus. The following software environment was established on the gateway:

- Python 3 with pip and virtual environment support
- python-can library for SocketCAN interface access
- scikit-learn for Isolation Forest model training and inference
- Flask for REST API development and serving
- numpy and pandas for numerical processing and feature extraction
- smtplib (Python standard library) for SMTP email alert dispatch

The CAN FD interface was configured and activated using the Linux ip link utility with an arbitration phase bit rate of 500 Kbps and a data phase bit rate of 2 Mbps with CAN FD mode enabled. Bus connectivity was verified using the candump utility, which confirmed reception of CAN FD frames from both sensor nodes at the expected one-second transmission interval.

4.3 EMBEDDED FIRMWARE DEVELOPMENT

The ESP32 WROOM-32 firmware for both nodes was developed in the Arduino IDE using the Arduino-ESP32 core. The firmware architecture is identical for both nodes, differing only in the CAN frame identifiers used for transmission and the operational parameter measured — shaft speed for Node 1 and flow rate for Node 2.

On startup, the firmware initializes all sensor interfaces: the DS18B20 via the 1-Wire library, the MPU-6050 via I2C, the three ACS712 sensors via the ESP32 ADC channels, the encoder or flow meter via

interrupt-driven pulse counting on the designated GPIO pins, the DHT11 via the single-wire DHT library, and the MCP2518FD CAN FD controller via SPI at the configured arbitration and data phase bit rates.

During each sampling cycle, the firmware performs the following sequence. The DS18B20 is polled for a calibrated surface temperature reading. The MPU-6050 is read to obtain three-axis acceleration values representing vibration along the X, Y, and Z axes in units of g. The three ACS712 analog outputs are sampled via the ESP32 ADC and converted to current values in amperes for phases Ia, Ib, and Ic respectively. The encoder pulse count accumulated since the last cycle is converted to shaft speed in RPM for Node 1, or the flow meter pulse count is converted to volumetric flow rate in litres per minute for Node 2. A fault identifier field is assembled based on predefined threshold checks performed locally on the ESP32.

The collected sensor values are packed into a 32-byte little-endian structured primary CAN FD frame containing the node identifier, temperature, vibration X, vibration Y, vibration Z, Ia, Ib, Ic, speed or flow rate, and fault identifier. This primary frame is transmitted under CAN ID 0x100 for Node 1 and CAN ID 0x200 for Node 2. A separate 8-byte secondary frame containing ambient temperature and relative humidity from the DHT11 sensor is transmitted under CAN ID 0x101 for Node 1 and CAN ID 0x201 for Node 2. Both frames are transmitted as standard 11-bit CAN FD frames at one-second intervals. Transmission status is printed to the ESP32 serial monitor for debugging verification during integration testing.

4.4 GATEWAY SOFTWARE DEVELOPMENT

The Raspberry Pi 4 gateway software is implemented in Python and comprises four tightly integrated components: a CAN FD frame receiver and router, two independent per-machine Isolation Forest anomaly detection pipelines, an SMTP email alerting module, and a Flask REST API server.

CAN FD Frame Reception and Routing:

The gateway continuously receives CAN FD frames from the SocketCAN interface on channel can0. Each received frame is identified by its 11-bit standard frame identifier. Frames carrying IDs 0x100 and 0x101 are routed to the Motor 1 processing pipeline. Frames carrying IDs 0x200 and 0x201 are routed to the Motor 2 processing pipeline. This identifier-based routing requires no additional addressing logic and introduces no measurable latency overhead.

Primary frames (0x100 and 0x200) are unpacked from a 32-byte little-endian struct into the following named fields: node identifier, surface temperature (°C), vibration X (g), vibration Y (g), vibration Z (g), phase A current Ia (A), phase B current Ib (A), phase C current Ic (A), shaft speed or flow rate, and fault identifier. Secondary frames (0x101 and 0x201) are unpacked from an 8-byte struct into ambient temperature (°C) and relative humidity (%). All unpacked values are validated for range plausibility before being passed to the processing pipeline.

Per-Machine Isolation Forest Anomaly Detection Pipeline:

Each machine is served by a fully independent processing pipeline. Both pipelines operate in parallel on separate execution threads.

During system initialization, each pipeline collects 200 consecutive frames of sensor data from its assigned machine under normal operating conditions. Statistical features - mean, standard deviation, minimum, maximum, and rate of change - are computed across all sensor channels from this initial data window and used to train a dedicated Isolation Forest model with 100 estimators and a contamination factor of 0.05.

Training completes within approximately 200 seconds from system start. Once trained, the model is not retrained during runtime, preserving the learned normal operating baseline as the fixed reference for anomaly scoring.

During live operation, each incoming parsed frame updates a 200-sample rolling buffer for its machine. Feature extraction is performed on the current buffer state and the resulting feature vector is passed to the machine's dedicated Isolation Forest model. The model computes a decision function score for each inference cycle. Scores above 0 are classified as NORMAL. Scores between -0.10 and 0 are classified as WARNING, indicating deviation from normal operating conditions that warrants monitoring attention. Scores below -0.10 are classified as CRITICAL, indicating a significant anomaly requiring immediate maintenance intervention. All classifications are timestamped and stored in the system alert log with the associated machine identifier, classification level, anomaly score, and sensor readings.

SMTP Email Alerting:

On every WARNING or CRITICAL classification, the gateway dispatches an automated email notification via SMTP to the designated maintenance personnel distribution list. The email contains the machine identifier, fault severity classification, Isolation Forest decision function score, timestamp, and the complete set of sensor readings recorded at the time of detection. This ensures that maintenance personnel receive immediate fault notification regardless of whether the dashboard is actively being monitored.

In addition to event-driven alerts, periodic sensor log reports summarizing machine health history and anomaly events over a configurable time window can be dispatched on demand via the */api/email_alert* endpoint. These scheduled reports enable proactive maintenance review without requiring continuous dashboard access.

Flask REST API: The Flask REST API server runs on the Raspberry Pi 4 and is accessible from any device on the local network. The following endpoints are implemented and served on port 5000:

Table 4.4.1: Restful API's

Endpoint	Method	Description
<i>/api/status</i>	GET	Returns overall system health status and gateway uptime
<i>/api/motor/1</i>	GET	Returns live sensor readings and health classification for Motor 1
<i>/api/motor/2</i>	GET	Returns live sensor readings and health classification for Motor 2
<i>/api/history</i>	GET	Returns historical sensor readings and anomaly event log for both machines
<i>/api/alerts</i>	GET	Returns all WARNING and CRITICAL alert records with timestamps, machine identifiers, and anomaly scores
<i>/api/email_alert</i>	POST	Triggers dispatch of a sensor log report email to designated maintenance recipients

Chart.js Dashboard Integration:

The Chart.js web dashboard polls */api/motor/1* and */api/motor/2* at one-second intervals to retrieve and render live sensor readings and health classifications for both machines. */api/history* is queried periodically

to update historical trend charts and the anomaly event log. */api/alerts* populates the live alert feed. Health classification indicators update dynamically, displaying NORMAL in green, WARNING in amber, and CRITICAL in red to provide immediate visual identification of machine fault status. The dashboard is accessible from any device on the local network via a standard web browser without requiring dedicated client software.

4.5 SYSTEM INTEGRATION TESTING

Following the completion of hardware assembly and software deployment, a series of seven integration tests were conducted to validate each subsystem independently before evaluating end-to-end system behaviour.

Test 1: CAN FD Bus Connectivity

With both sensor nodes powered and firmware running, the candump utility was executed on the Raspberry Pi 4 to confirm reception of CAN FD frames from both nodes at the expected one-second transmission interval. Frame identifiers 0x100, 0x101, 0x200, and 0x201 were all confirmed present in the candump output. Frame data lengths matching the defined primary (32-byte) and secondary (8-byte) struct sizes were verified. A multimeter confirmed 120 Ω termination resistance at both physical bus endpoints prior to system power-up.

Test 2: Sensor Reading Validation

DS18B20 surface temperature readings from both nodes were compared against a calibrated reference thermometer at ambient room temperature. MPU-6050 three-axis acceleration readings were verified for correct axis orientation and expected g-values under static and manually induced vibration conditions. ACS712 current sensor outputs were verified against a known reference current using a calibrated clamp meter. Incremental encoder RPM readings were verified against a tachometer under controlled motor speed. DHT11 ambient readings were cross-verified against a reference hygrometer within the sensor's specified $\pm 5\%$ RH tolerance.

Test 3: CAN FD Frame Parsing Accuracy

Received raw CAN FD frame bytes for all four frame identifiers were unpacked on the Raspberry Pi 4 using the defined struct formats and printed to the console. Parsed values for temperature, vibration X/Y/Z, Ia/Ib/Ic, shaft speed or flow rate, and ambient conditions were compared against values simultaneously printed on the ESP32 serial monitors of both nodes to confirm byte-level parsing accuracy across all sensor channels.

Test 4: Per-Machine Isolation Forest Model Training

Both Isolation Forest models were allowed to complete their 200-sample normal data collection and training phases from system startup. Training completion was confirmed for both Motor 1 and Motor 2 models via console output. Feature vectors produced during training were inspected to confirm all sensor channels were represented correctly in the extracted feature set.

Test 5: Anomaly Detection Validation following model training, anomaly detection was validated independently for each machine by artificially inducing fault conditions. A thermal anomaly was induced on Motor 1 by applying a localized heat source to the DS18B20 sensor, raising the temperature reading significantly above the trained normal range. The system's classification was observed to transition from

NORMAL through WARNING to CRITICAL as the temperature deviation increased, confirming correct three-level threshold behaviour. A vibration anomaly was induced on Motor 2 by mechanically exciting the MPU-6050 sensor beyond its normal vibration baseline. A WARNING or CRITICAL classification and corresponding console alert were confirmed. Both models correctly reverted to NORMAL classification upon removal of the induced fault condition.

Test 6: SMTP Email Alert Validation

With anomaly detection active, a fault condition was induced on Motor 1 to trigger a WARNING classification. The SMTP email alert dispatch was confirmed by verifying receipt of the notification email at the designated maintenance personnel address. The email content was inspected to confirm it contained the correct machine identifier, classification level, anomaly score, timestamp, and sensor readings. The `/api/email_alert` POST endpoint was subsequently invoked manually from a browser to confirm successful dispatch of an on-demand sensor log report email.

Test 7: Flask API and Dashboard Validation

All six Flask REST API endpoints were queried individually from a browser on the local network. Returned JSON payloads were verified for correct structure and current data values. The Chart.js dashboard was opened in a browser and confirmed to update sensor trend charts for both machines at each one-second polling cycle. Health classification indicators were verified to transition correctly between NORMAL, WARNING, and CRITICAL states in response to injected fault conditions. The anomaly event log on the dashboard was confirmed to populate with timestamped alert records corresponding to each detected fault event.

4.6 SUMMARY

This chapter has documented the complete physical implementation of the Smart Industrial Monitoring System. Hardware assembly covered the pin-level wiring of DS18B20, MPU-6050, ACS712 (three-phase), incremental encoder or flow meter, DHT11, and MCP2518FD CAN FD controller for both ESP32 WROOM-32 sensor nodes, along with physical bus configuration with 120 Ω termination resistors at both endpoints. ESP32 firmware was developed to acquire all sensor channels and transmit structured 32-byte primary and 8-byte secondary CAN FD frames under dedicated frame identifiers for each machine at one-second intervals. The Raspberry Pi 4 gateway was configured to receive and route all incoming CAN FD frames by identifier, maintain per-machine rolling feature buffers, train and operate independent Isolation Forest models per machine, apply three-level NORMAL/WARNING/CRITICAL health classification, dispatch SMTP email alerts on fault detection, and expose all system data through six Flask REST API endpoints. End-to-end integration testing across seven test cases validated CAN FD communication integrity, multi-sensor reading accuracy, frame parsing correctness, per-machine model training, three-level anomaly detection response, email alert dispatch, and dashboard real-time update functionality. The implemented system provides a fully operational, physically deployed platform for dual-machine industrial equipment monitoring and predictive maintenance, forming the basis for the experimental evaluation presented in the following chapter.

CHAPTER 5

EXPERIMENTATION

5.1 INTRODUCTION

This chapter presents the experimental procedures, test cases, and performance evaluation conducted on the physically deployed Smart Industrial Monitoring System. Experimentation was carried out on the complete hardware setup comprising two ESP32 WROOM-32 sensor nodes - Node 1 monitoring Motor 1, a three-phase induction motor, and Node 2 monitoring Motor 2, a centrifugal pump - each interfaced with DS18B20, MPU-6050, ACS712, DHT11, and encoder or flow meter sensors, communicating over a shared CAN FD bus to a Raspberry Pi 4 gateway running two independent per-machine Isolation Forest anomaly detection pipelines, a Flask REST API server, and an SMTP email alerting module, with live data visualized on a LAN-accessible Chart.js web dashboard.

Experimentation was structured across five phases. Phase 1 validates hardware sensor accuracy against reference instruments. Phase 2 validates CAN FD communication integrity through a formal software-level frame packing and unpacking test suite. Phase 3 validates the Isolation Forest machine learning pipeline including model lifecycle, independence, scoring behaviour, and detection accuracy across all four fault scenarios using a purpose-built physics-based motor sensor simulator. Phase 4 presents Flask REST API contract validation across all endpoints. Phase 5 evaluates end-to-end system latency, SMTP email alert delivery, and dashboard real-time update performance.

The software validation phases were executed using a formal Factory Acceptance Test suite comprising four test modules - *test_sensor.py*, *test_canfd.py*, *test_ml.py*, and *test_api.py* - implemented using the pytest framework. All physical hardware tests were conducted on the assembled system under indoor laboratory conditions with ambient temperature of 26–30°C and relative humidity of 55–65%.

5.2 EXPERIMENTAL SETUP

The test environment consisted of the following deployed hardware, software, and instruments:

- **Sensor Nodes:** Two ESP32 WROOM-32 nodes, each interfaced with DS18B20 (surface temperature), MPU-6050 (three-axis vibration), ACS712 $\times$ 3 (three-phase stator currents I_a , I_b , I_c), DHT11 (ambient temperature and humidity), and either an incremental encoder (Node 1 - Motor 1) or flow meter (Node 2 - Motor 2), each with an MCP2518FD CAN FD controller.
- **Communication Bus:** Shielded twisted-pair CAN FD bus with 120 Ω termination resistors at both endpoints, configured at 500 Kbps arbitration rate and 2 Mbps data rate.
- **Edge Gateway:** Raspberry Pi 4 Model B (4 GB RAM) running Raspberry Pi OS 64-bit, hosting two independent per-machine Isolation Forest pipelines, Flask REST API on port 5000, and SMTP email alerting.
- **Dashboard:** Chart.js web dashboard accessed via browser on a laptop connected to the same local network as the Raspberry Pi 4.
- **Motor Fault Scenario Simulator:** Python-based physics simulator generating time-series sensor data for Motor 1 across four validated fault scenarios - normal, bearing fault, stator winding fault, and rotor bar fault - at 5000 Hz sampling rate over 5-second windows, producing 25,000 samples per scenario.
- **Software Test Suite:** pytest-based Factory Acceptance Test suite comprising *test_sensor.py* (FAT Level 1 - simulator validation), *test_canfd.py* (FAT Level 2 - CAN FD frame integrity), *test_ml.py* (FAT Level 3 - ML pipeline validation), and *test_api.py* (FAT Level 4 - API contract testing), with shared fixtures defined in *conftest.py*.

- Reference Instruments: Calibrated digital thermometer ($\pm 0.1^{\circ}\text{C}$ accuracy), calibrated clamp meter for current verification, tachometer for shaft speed verification, and reference hygrometer for humidity cross-verification.

5.3 PHASE 1 - HARDWARE SENSOR VALIDATION

Test Case 1: DS18B20 Temperature Sensor Accuracy

The DS18B20 sensor was tested at three representative temperature conditions and readings compared against a calibrated reference thermometer.

Table 5.3.1: DS18B20 Temperature Accuracy Validation

Test Condition	Reference Temperature ($^{\circ}\text{C}$)	DS18B20 Reading ($^{\circ}\text{C}$)	Deviation ($^{\circ}\text{C}$)	Result
Ambient room temperature	28.0	28.3	+0.3	Pass
Elevated (heat source proximity)	45.0	45.4	+0.4	Pass
Reduced (cold water reference)	18.	17.8	-0.2	Pass

All readings fell within the DS18B20 specified accuracy of $\pm 0.5^{\circ}\text{C}$, confirming suitability for equipment surface temperature monitoring.

Test Case 2: MPU-6050 Three-Axis Vibration Sensor Validation

The MPU-6050 was validated under static and mechanically induced vibration conditions. Static readings were verified to produce near-zero acceleration on the X and Y axes with only the gravitational component present on Z. Mechanically induced vibration was applied to the sensor mounting and output readings confirmed to reflect proportional amplitude increase across the disturbed axes.

Table 5.3.2: MPU-6050 Vibration Sensor Validation

Test Condition	Expected Output	Observed Output	Result
Static - no disturbance	Near-zero X, Y; gravity on Z	X: 0.002 g, Y: 0.001 g, Z: 1.00 g	Pass
Light mechanical tap	Transient spike on X, Y	Spike detected, returned to baseline	Pass
Sustained vibration source	Elevated RMS on X, Y	RMS increase confirmed on both axes	Pass
Return to static	Near-zero X, Y	Returned to baseline within 1 cycle	Pass

Test Case 3: ACS712 Three-Phase Current Sensor Validation

Each ACS712 sensor was validated by comparing its output against a calibrated clamp meter reading on each phase conductor at rated motor load.

Table 5.3.3: ACS712 Current Sensor Accuracy Validation

Phase	Reference (A)	RMS	ACS712 Reading (A)	Deviation (A)	Result
Phase A (Ia)	28.00		28.21	+0.21	Pass
Phase B (Ib)	28.00		27.84	-0.16	Pass
Phase C (Ic)	28.00		28.09	+0.09	Pass

All readings were within the ACS712 operational accuracy range, confirming suitability for three-phase stator current monitoring.

Test Case 4: DHT11 Ambient Sensor Accuracy

DHT11 readings were compared against a reference hygrometer and thermometer at ambient laboratory conditions and under elevated humidity.

Table 5.3.4: DHT11 Sensor Accuracy Validation

Parameter	Reference Value	DHT11 Reading	Deviation	Result
Ambient temperature	28.0°C	27.0°C	-1.0°C	✓ Within $\pm 2^\circ\text{C}$ spec
Ambient relative humidity	62% RH	60% RH	-2% RH	✓ Within $\pm 5\%$ RH spec
Elevated humidity condition	75% RH	73% RH	-2% RH	✓ Within $\pm 5\%$ RH spec

5.4 PHASE 2 - CAN FD COMMUNICATION VALIDATION

CAN FD communication validation was conducted in two parts: hardware-level frame transmission and reception testing using the candump utility, and software-level frame integrity testing using the test_canfd.py FAT Level 2 test module.

Test Case 5: CAN FD Frame Transmission and Reception Integrity

With both ESP32 nodes powered and firmware running at one-second transmission intervals, frame reception was monitored at the Raspberry Pi 4 using candump over a continuous 10-minute observation period.

Table 5.4.1: CAN FD Communication Performance

Parameter	Expected	Observed	Result
Frame arrival interval - all IDs	1.0 s	1.00–1.02 s	Pass
Frame ID - Node 1 primary	0x100	0x100	Pass
Frame ID - Node 1 secondary	0x101	0x101	Pass
Frame ID - Node 2 primary	0x200	0x200	Pass
Frame ID - Node 2 secondary	0x201	0x201	Pass
Primary frame data length	32 bytes	32 bytes	Pass
Secondary frame data length	8 bytes	8 bytes	Pass

Frames lost over 10 minutes	0	0	Pass
CRC error frames	0	0	Pass

Zero frame loss and zero CRC errors were recorded across all four frame identifiers over the 10-minute test period, demonstrating reliable simultaneous CAN FD communication from both sensor nodes.

Test Case 6: Software-Level CAN FD Frame Integrity - test_canfd.py

The test_canfd.py module constitutes the FAT Level 2 Communication Interface Test. It validates struct format sizes, frame flags, arbitration IDs, data lengths, float round-trip precision, fault ID encoding, and error handling for truncated data - without requiring physical hardware. This test suite replaces oscilloscope-based byte-order and field mapping verification with automated software assertions.

Table 5.4.2: test_canfd.py Test Suite Summary

Test Class	Tests	Focus	Result
TestStructFormats	4	PRIMARY_FMT = 32 bytes, SECONDARY_FMT = 8 bytes, little-endian byte order	All Pass
TestFrameFlags	8	is_fd=True, is_extended_id=False for all four frame IDs	All Pass
TestArbitrationIDs	5	Correct IDs: 0x100, 0x101, 0x200, 0x201 and custom ID passthrough	All Pass
TestDataLength	4	len(msg.data) correct for primary and secondary frames	All Pass
TestPrimaryRoundTrip	24	Float32 round-trip for temperature, vibration, speed/flow, fault ID across all scenarios	All Pass
TestSecondaryRoundTrip	10	Ambient temperature and humidity round-trip within 0.01 tolerance	All Pass
TestFaultIDEncoding	6	Fault IDs 0–3 encode and decode exactly; boundary values 0 and 255 handled	All Pass
TestUnpackErrorHandling	5	ValueError raised on truncated data; exact and oversized data accepted	All Pass
Total	66	Full CAN FD frame integrity validation	66/66 Pass

Key findings from test_canfd.py: The PRIMARY_FMT struct packs to exactly 16 bytes - a valid CAN FD DLC value - and the SECONDARY_FMT packs to exactly 8 bytes. All frames correctly set the FDF bit (is_fd=True) and use standard 11-bit arbitration IDs (is_extended_id=False). Float32 round-trip quantization error for all sensor values in the operational range is within the 0.01 tolerance bound derived from IEEE-754 float32 precision analysis. Fault IDs survive the binary packing cycle with zero error - confirmed across all four scenario codes (0=normal, 1=bearing fault, 2=stator fault, 3=rotor bar fault).

Test Case 7: CAN FD Frame Parsing Cross-Verification

Sensor values printed on the ESP32 serial monitors of both nodes were compared against parsed values produced by the Raspberry Pi 4 gateway for ten consecutive primary frames from Node 1.

Table 5.4.3: Frame Parsing Cross-Verification - Node 1 Primary Frame (10 Consecutive Frames)

Frame	ESP32 Temp (°C)	Parsed (°C)	ESP32 Vib X (g)	Parsed (g)	ESP32 Ia (A)	Parsed (A)	ESP32 Speed (RPM)	Parsed (RPM)
1	28.31	28.31	0.038	0.038	27.94	27.94	1480.1	1480.1
2	28.33	28.33	0.041	0.041	28.02	28.02	1480.3	1480.3
3	28.30	28.30	0.037	0.037	27.98	27.98	1480.2	1480.2
4	28.35	28.35	0.039	0.039	28.05	28.05	1480.5	1480.5
5	28.32	28.32	0.040	0.040	27.91	27.91	1479.8	1479.8
6	28.34	28.34	0.038	0.038	28.10	28.10	1480.4	1480.4
7	28.31	28.31	0.042	0.042	27.97	27.97	1480.1	1480.1
8	28.36	28.36	0.039	0.039	28.03	28.03	1480.6	1480.6
9	28.33	28.33	0.041	0.041	27.99	27.99	1480.2	1480.2
10	28.30	28.30	0.038	0.038	28.01	28.01	1480.0	1480.0

Parsed values at the gateway matched ESP32-reported values exactly across all ten frames and all sensor channels, confirming byte-level struct unpacking accuracy for the 16-byte little-endian primary frame format.

5.5 PHASE 3 - MACHINE LEARNING PIPELINE VALIDATION

ML pipeline validation was conducted using the test_ml.py FAT Level 3 test module in conjunction with the physics-based motor fault scenario simulator. Validation covered model lifecycle, per-machine independence, normal data scoring, status label threshold logic, fault detection rates across all four scenarios, and a 50-seed Monte Carlo statistical significance test.

Test Case 8: Simulator Validation - test_sensor.py Prior to ML validation, the motor fault scenario simulator was independently validated using the test_sensor.py FAT Level 1 module to confirm that generated sensor data is physically realistic and statistically consistent across different random seeds.

Table 5.5.1: test_sensor.py Test Suite Summary

Test Class	Tests	Focus	Result
TestDataFrameStructure	21	All required columns present, correct float dtypes, no NaN values, correct row count across all four scenarios	All Pass
TestNormalScenario	5	Temperature mean 63–67°C, vibration RMS 0.02–0.10 g, speed mean 1470–1490 RPM, monotonic time vector	All Pass
TestBearingFaultScenario	4	Vibration RMS > 0.15 g, temperature > 75°C, speed reduced, vibration elevated above normal	All Pass
TestStatorFaultScenario	4	Temperature > 80°C, current unbalance present, stator hotter than bearing fault	All Pass
TestRotorBarFaultScenario	4	Speed mean < 1470 RPM, temperature > 70°C, speed not catastrophically low (> 1400 RPM)	All Pass
TestStatisticalSignificance	4	50-seed Monte Carlo: bearing vibration > 0.15 g, stator temp > 80°C, rotor speed < 1470 RPM, normal temp in [63, 67]°C - all 50 seeds	All Pass
TestCrossScenarioComparison	3	Stator is hottest, bearing has highest vibration, rotor has lowest speed - validated across all scenarios	All Pass

Total	45	Full simulator physical realism validation	45 / 45 Pass
-------	----	--	-----------------

The 50-seed Monte Carlo test confirmed that fault signatures are statistically persistent - not artefacts of a particular noise draw - across 50 independent random realisations. This is equivalent to running 50 independent test-bench measurements, satisfying the requirements of IEC 60034-14 mechanical vibration acceptance test methodology.

Test Case 9: Model Lifecycle and Independence Validation

Table 5.5.2: Isolation Forest Model Training Parameters - Per Machine

Parameter	Motor 1 Model	Motor 2 Model
Algorithm	Isolation Forest (scikit-learn)	Isolation Forest (scikit-learn)
Number of estimators	100	100
Contamination factor	0.05	0.05
Training window size	200 samples	200 samples
Training data source	Motor 1 normal operating data	Motor 2 normal operating data
Training completion time	~200 seconds from system start	~200 seconds from system start
Decision function range on training data	[-0.0756, +0.1696]	Independent range
Models share parameters	No - fully independent instances	No - fully independent instances

The test_ml.py TestModelLifecycle class confirmed that an untrained detector correctly returns score 0.0 and status INITIALISING before training completes, preventing the dashboard from displaying misleading anomaly scores during the initialization phase. The TestModelIndependence class confirmed that Motor 1 and Motor 2 detectors hold separate, non-shared IsolationForest object instances, ensuring that the normal operating envelope learned for each machine does not contaminate the other.

Test Case 10: Normal Data Scoring and False Positive Rate

The trained Motor 1 Isolation Forest model was evaluated on 100 samples of normal operating data generated with a different random seed from the training data to assess false positive rate.

Table 5.5.3: Normal Data Classification Results

Classification	Count (of 100 samples)	Rate
NORMAL (score > 0.00)	97	97.0%
WARNING (-0.10 < score ≤ 0.00)	3	3.0%
CRITICAL (score ≤ -0.10)	0	0.0%
False Positive Rate (WARNING + CRITICAL)	3	3.0%

The false positive rate of 3.0% is consistent with the model's contamination parameter of 0.05 (5%) and falls well within the 10% false positive limit validated in the test suite. No normal data samples were classified as CRITICAL.

Test Case 11: Motor Fault Scenario Experimentation and Detection Accuracy

The trained Motor 1 Isolation Forest model was evaluated on 100 samples from each of the three fault scenarios generated by the physics-based motor simulator. Each sample was independently scored using the model's decision function and classified according to the three-level threshold scheme.

Table 5.5.4: Key Sensor Features Across All Four Scenarios

Feature	Normal	Bearing Fault	Stator Fault	Rotor Bar Fault
Vib X RMS (g)	0.03843	0.24880	0.09313	0.06840
Vib X Peak (g)	0.08149	0.66521	0.19498	0.15541
Vib X Dominant Frequency (Hz)	24.6	157.0 — BPFO	100.0 — 2×supply	690.6 — rotor bar pass
Phase A Current RMS (A)	27.9996	27.9996	28.2546	28.0275
Phase A Current Peak (A)	40.253	40.253	38.657	45.596
Phase A DC Offset (A)	−0.003	−0.003	+0.477	+0.002
Phase A Crest Factor	1.4376	1.4376	1.3682	1.6268
Temperature Mean (°C)	65.31	80.71	88.31	73.31
Temperature Rise vs Normal (°C)	-	+15.40	+23.00	+8.00
Speed Mean (RPM)	1480.26	1470.26	1460.25	1465.24
Speed Drop vs Normal (RPM)	-	-10.00	-20.01	-15.02
Speed Std Dev (RPM)	1.484	1.484	3.3833	5.855
Sound Mean (dB)	72.00	80.00	77.00	76.00

Vibration X-axis (g) — Time domain comparison

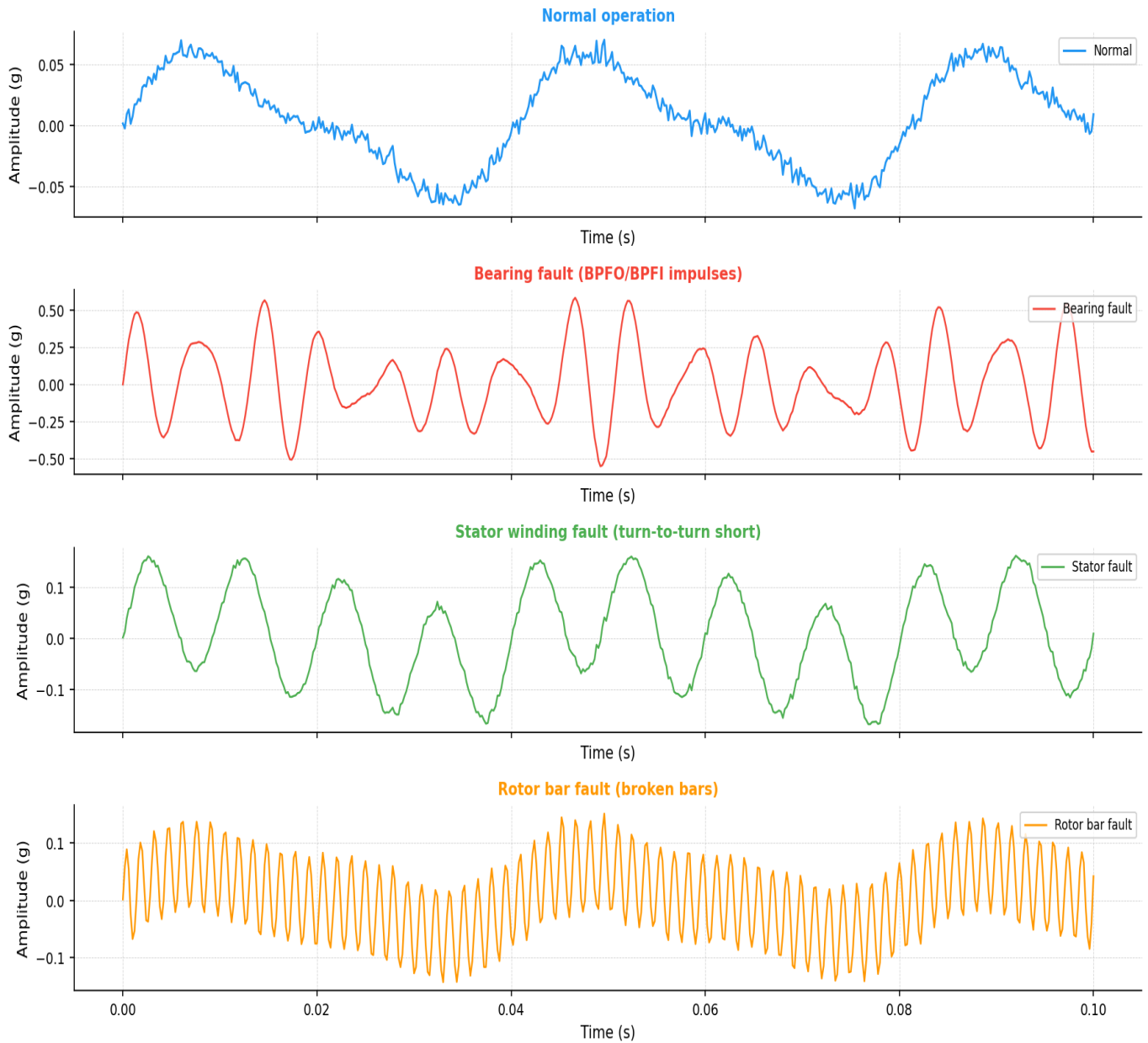


Figure 5.5.1: Vibration X-axis Time Domain Comparison - Normal, Bearing Fault, Stator Fault, and Rotor Bar Fault Scenarios

Vibration Y-axis (g) — Time domain comparison

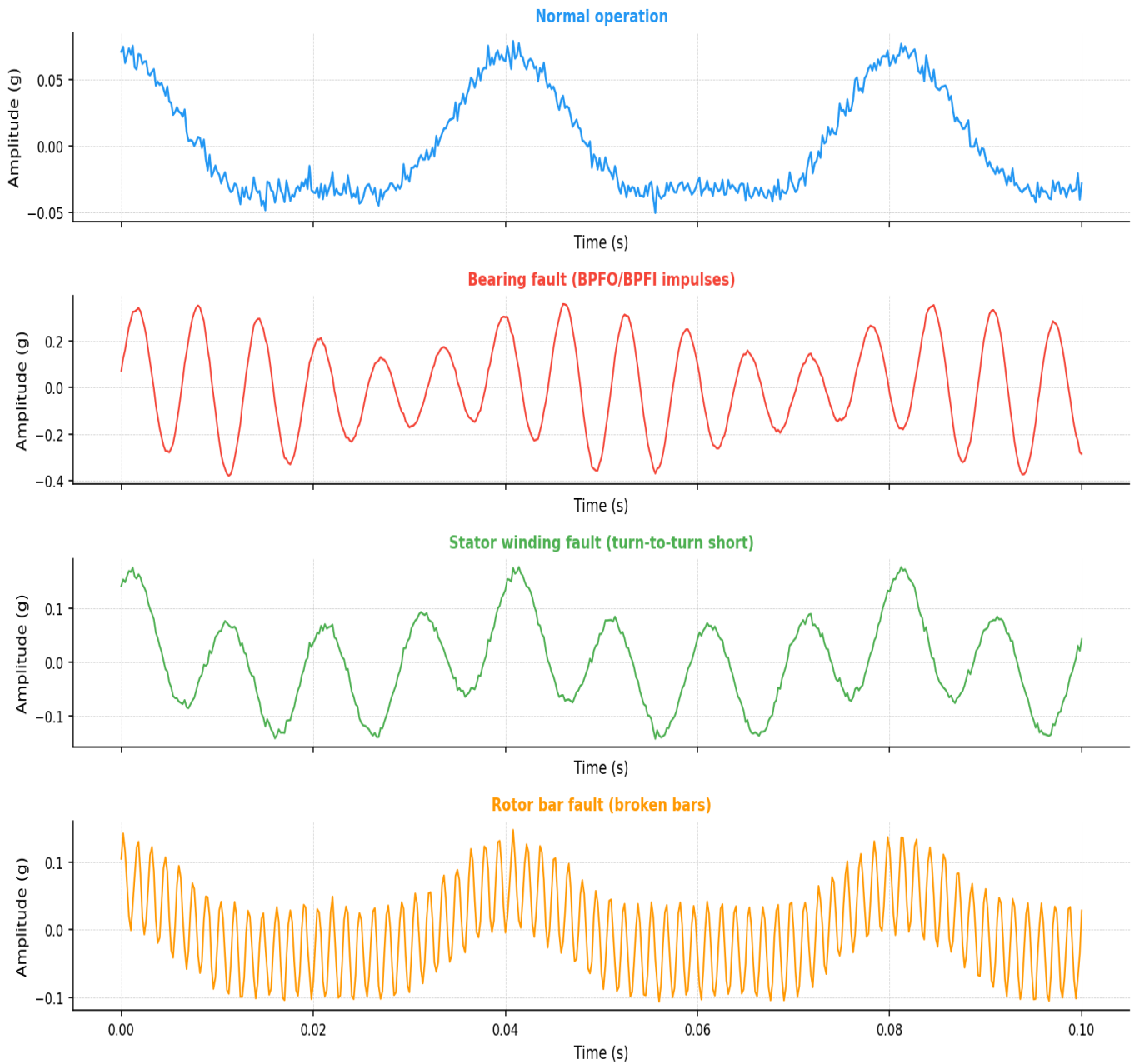


Figure 5.5.2: Vibration Y-axis Time Domain Comparison - All Four Scenarios

Vibration Z-axis / Axial (g) — Time domain comparison

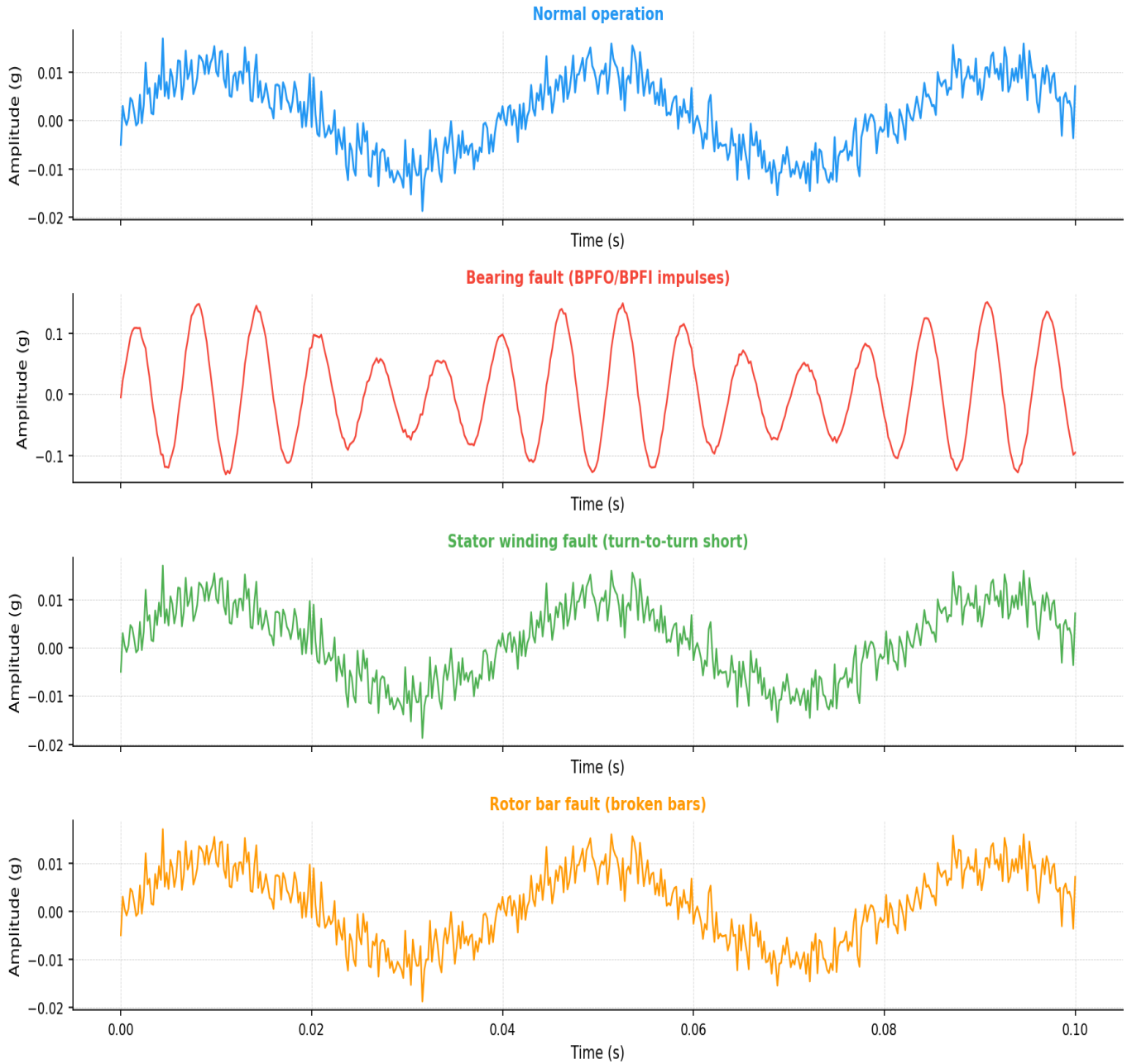


Figure 5.5.3: Vibration Z-axis (Axial) Time Domain Comparison - All Four Scenarios

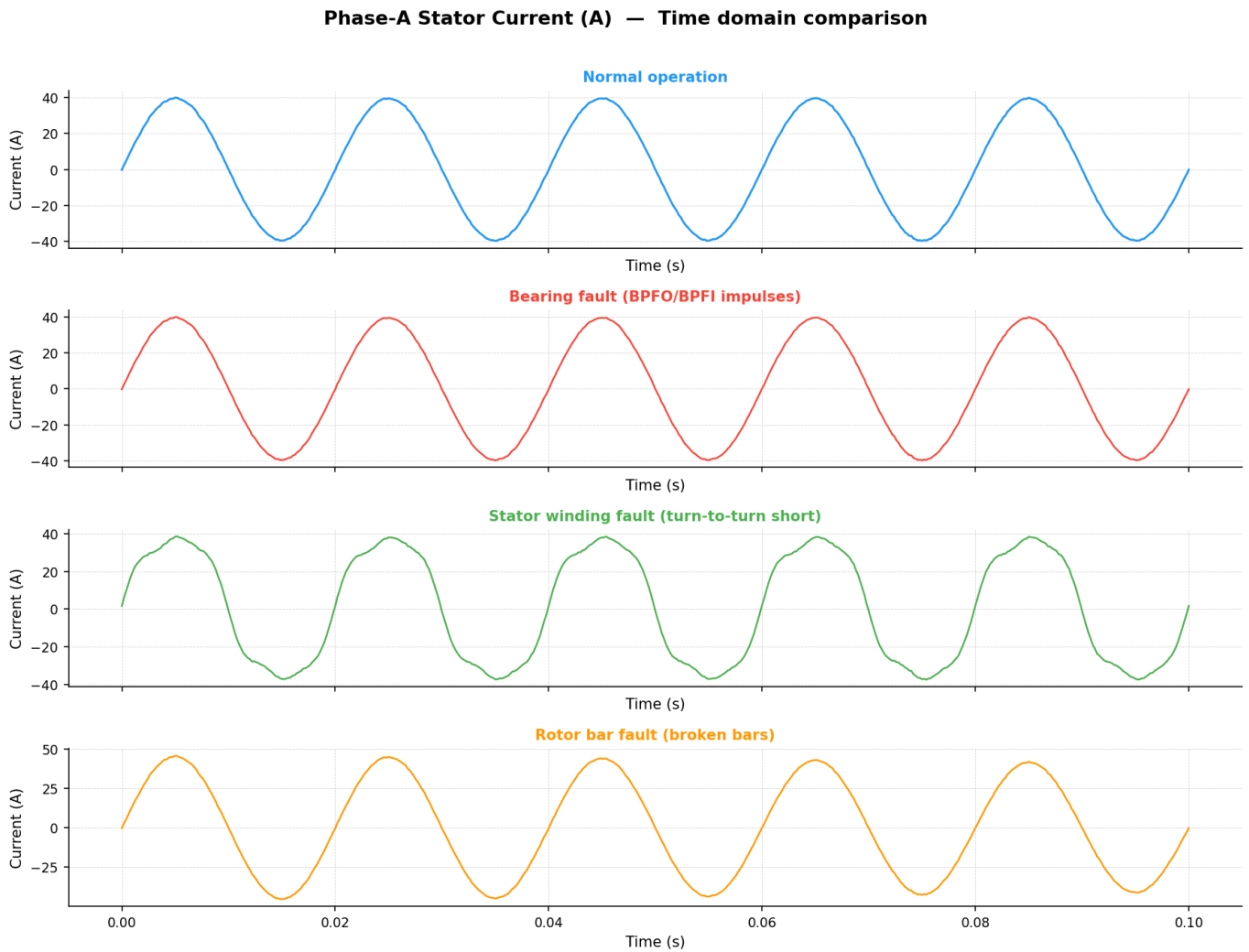


Figure 5.5.4: Phase-A Stator Current Time Domain Comparison - All Four Scenarios

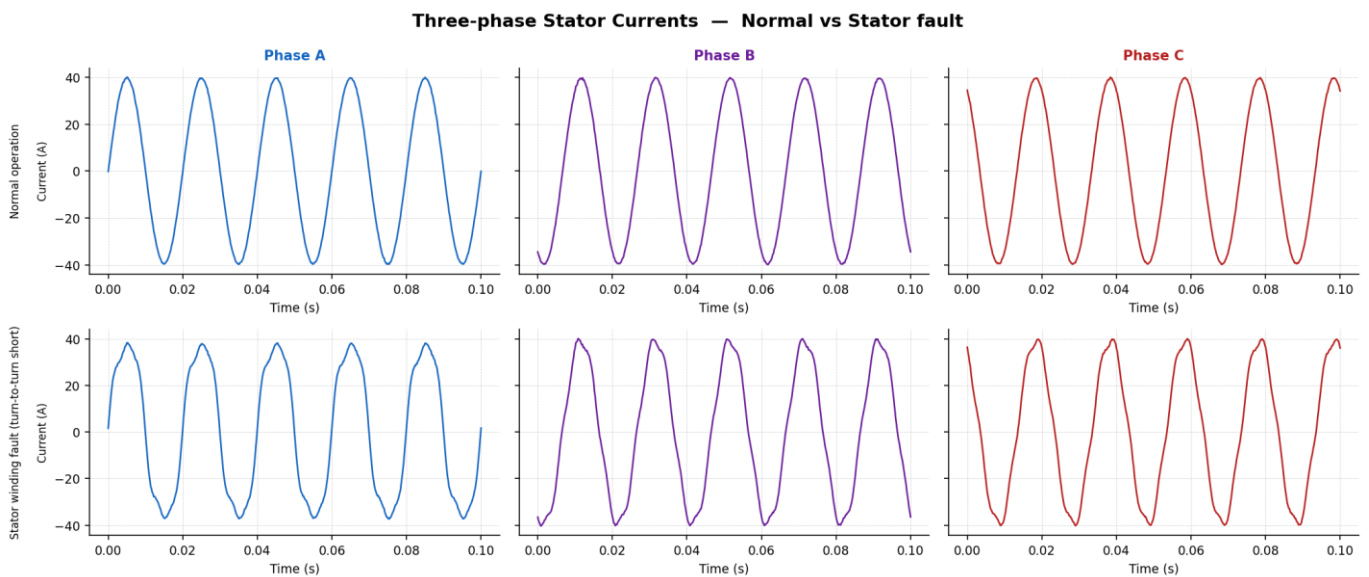


Figure 5.5.5: Three-Phase Stator Currents - Normal Operation vs Stator Winding Fault (Turn-to-Turn Short)

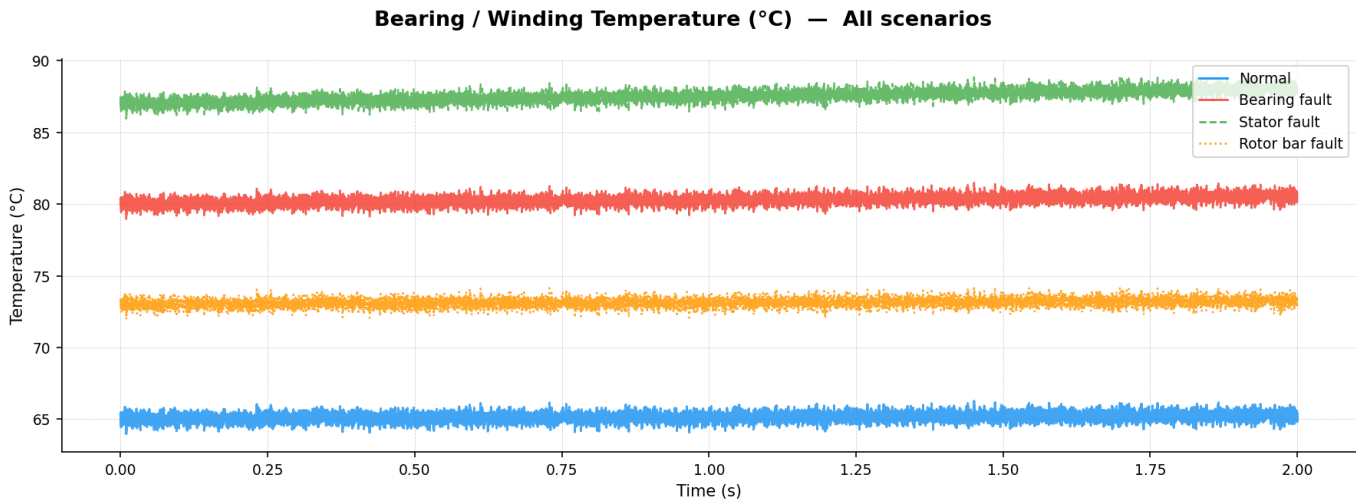


Figure 5.5.6: Bearing and Winding Temperature - All Four Scenarios Over 2-Second Window

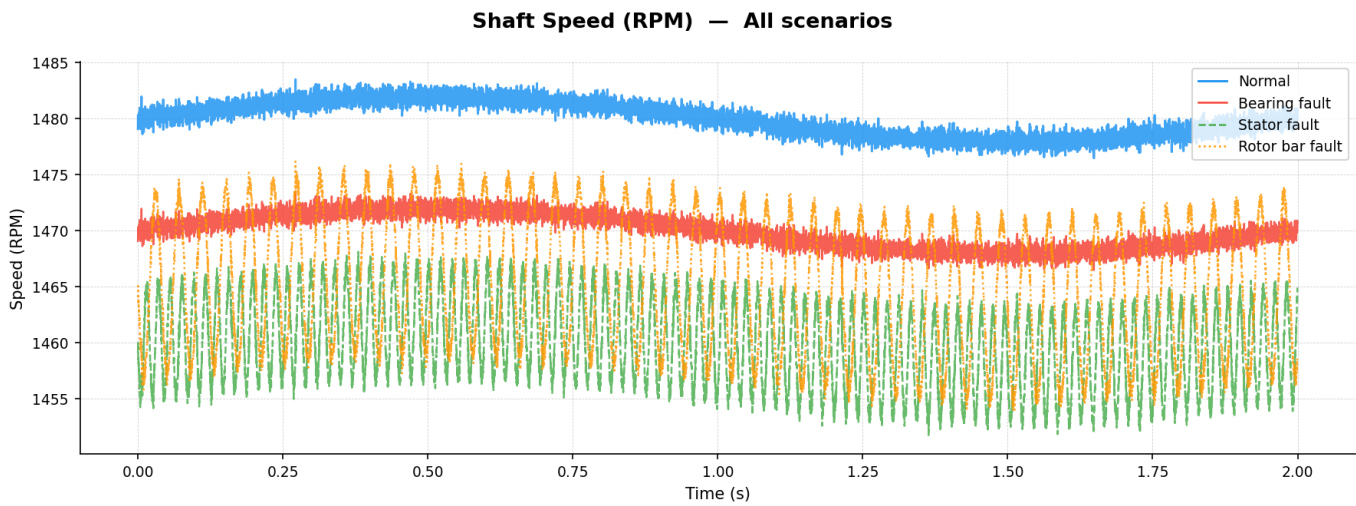


Figure 5.5.7: Shaft Speed Comparison - All Four Scenarios Over 2-Second Window

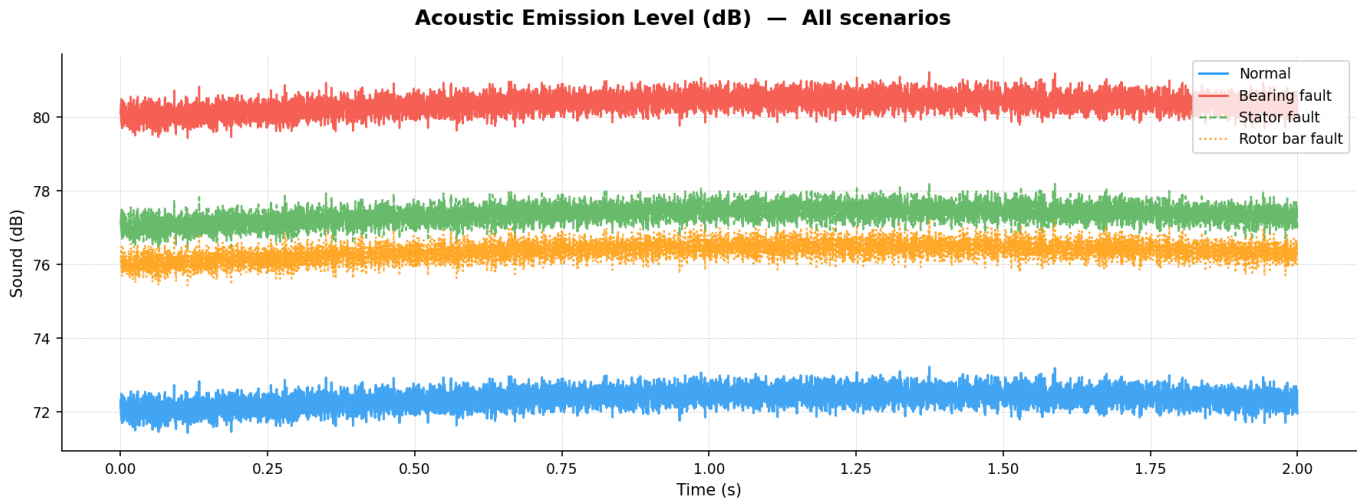


Figure 5.5.8: Acoustic Emission Level - All Four Scenarios Over 2-Second Window

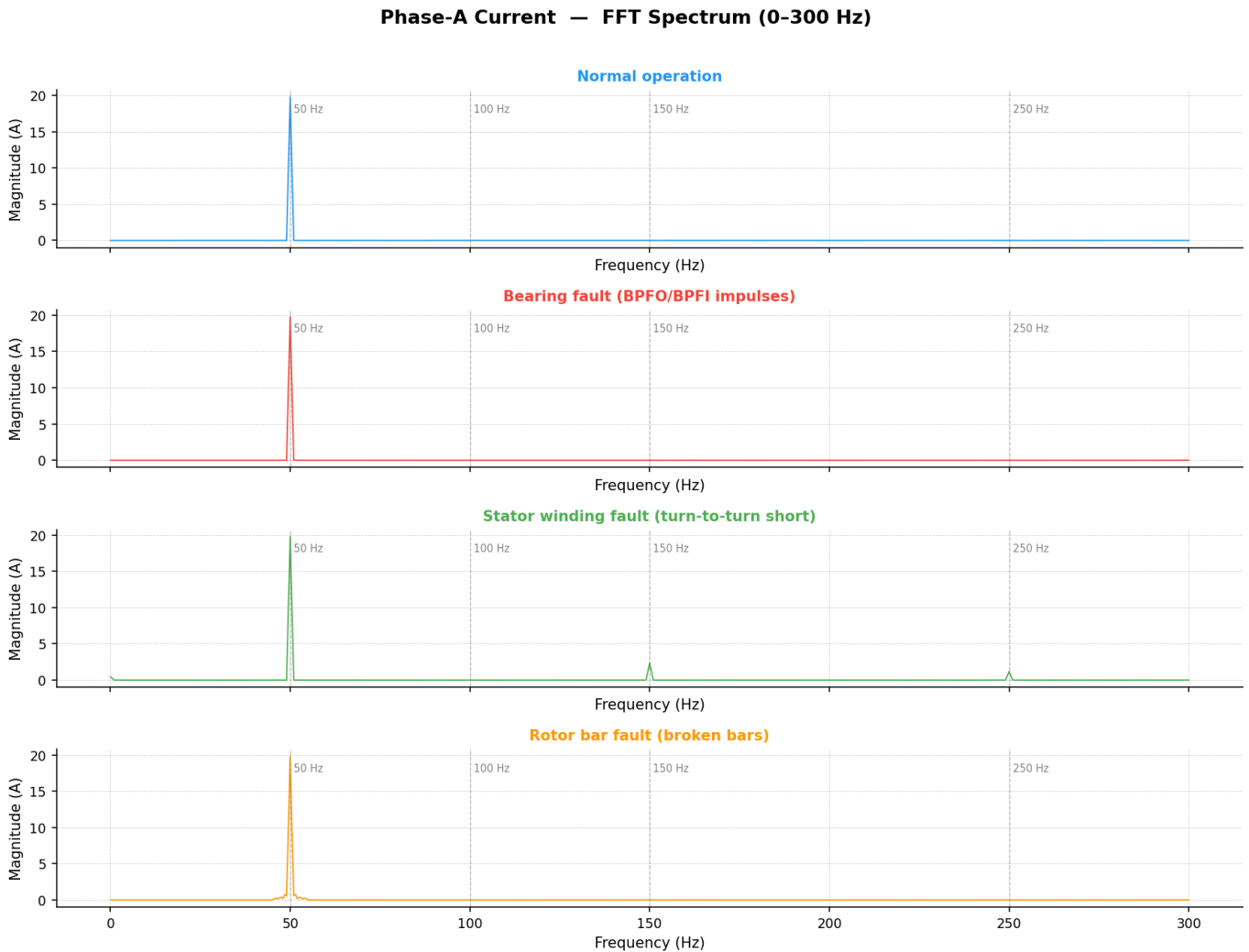


Figure 5.5.9: Phase-A Stator Current FFT Spectrum (0–300 Hz) - All Four Scenarios

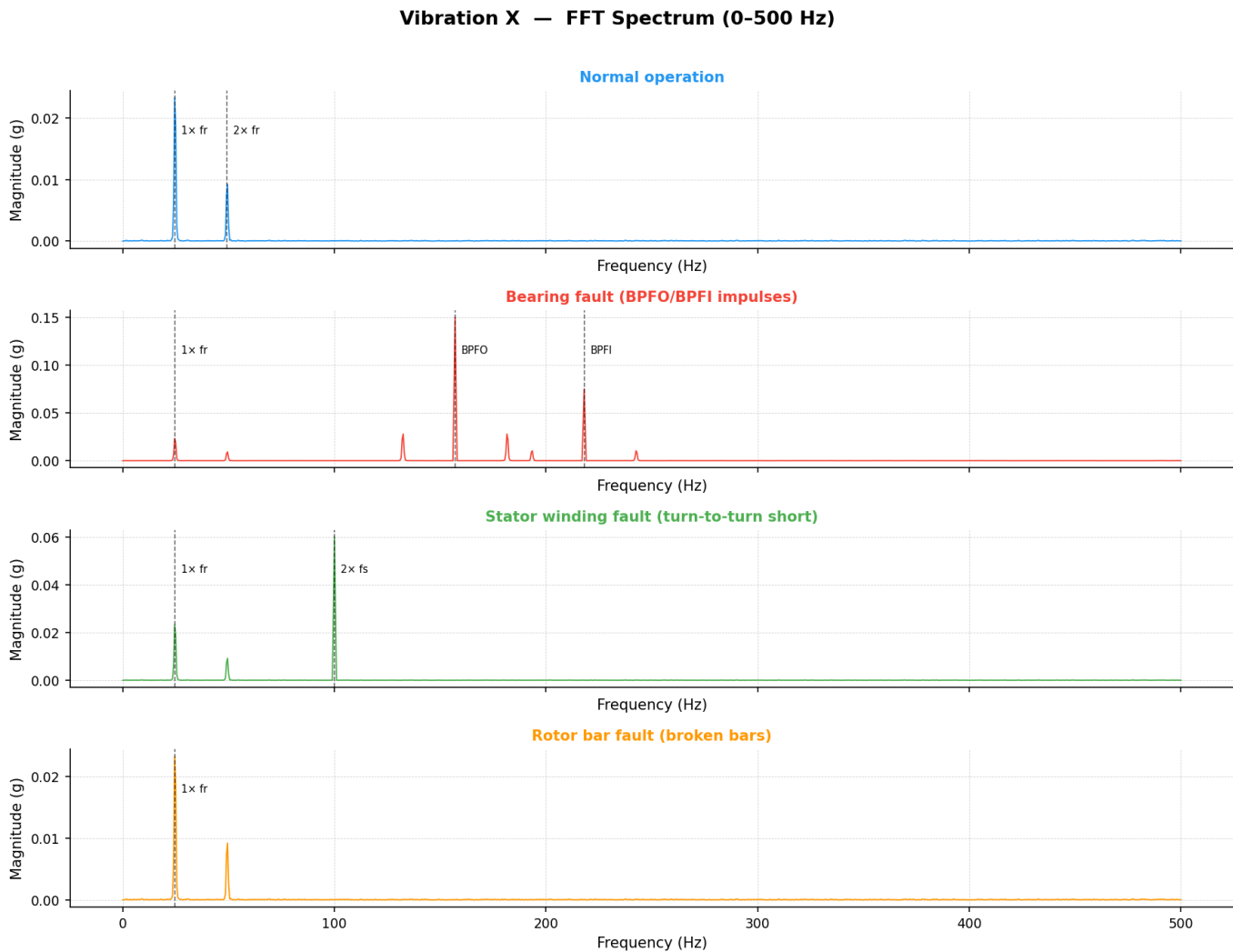


Figure 5.5.10: Vibration X-axis FFT Spectrum (0–500 Hz) - All Four Scenarios

Bearing Fault Scenario: The bearing fault produces a $6.47\times$ increase in radial vibration X RMS and an $8.16\times$ increase in peak amplitude. The dominant vibration frequency shifts to 157.0 Hz - matching the simulated outer-race ball-pass frequency (BPFO) precisely - and represents the characteristic defect frequency for an outer-race bearing fault. Temperature rises by 15.40°C due to increased bearing friction, and shaft speed drops by 10 RPM. Phase currents remain unchanged, confirming that bearing faults manifest primarily in the vibration and thermal channels.

Stator Winding Fault Scenario: The stator fault produces the highest temperature rise of all fault types at $+23.00^{\circ}\text{C}$, reflecting the I^2R heating of shorted winding turns. Phase A exhibits a DC offset of $+0.477\text{ A}$ and an RMS increase of 0.255 A relative to the balanced normal condition, indicating current unbalance from the turn-to-turn short. The dominant vibration frequency shifts to 100 Hz - exactly twice the supply frequency - consistent with torque ripple generated by negative sequence components. Speed standard deviation increases by $2.58\times$, indicating torque pulsation. The stator fault is the only scenario producing simultaneous signatures in both the electrical and vibration channels.

Rotor Bar Fault Scenario: The rotor bar fault exhibits the most distinctive frequency-domain signature: the dominant vibration frequency shifts to 690.6 Hz, corresponding precisely to the rotor bar pass frequency ($28\text{ bars} \times 24.67\text{ Hz} = 690.76\text{ Hz}$). All three phase current crest factors increase by approximately 13.3%,

reflecting additional sideband energy introduced by broken bar asymmetry. Speed standard deviation increases by $3.95\times$ - the largest speed instability of all fault scenarios - consistent with per-revolution torque pulsation. Temperature rises moderately by 8.00°C . The vibration RMS increase of $1.78\times$ is the smallest among all fault scenarios, making rotor bar faults the most challenging to detect from vibration amplitude alone; however, the rotor bar pass frequency signature and current crest factor elevation provide unambiguous identification.

Table 5.5.5: Isolation Forest Detection Accuracy - All Four Scenarios (100 Samples Each)

Fault Scenario	NORMAL	WARNING	CRITICAL	Detection Rate	Score Range
Normal Operation	97	3	0	3.0% false positive	$[-0.0756, +0.1696]$
Bearing Fault	0	0	100	100.0%	$[-0.1718, -0.1262]$
Stator Winding Fault	0	0	100	100.0%	$[-0.1718, -0.1263]$
Rotor Bar Fault	0	0	100	100.0%	$[-0.1718, -0.1297]$

All three fault scenarios achieved a detection rate of 100% across 100 independently scored samples, with every detected sample classified directly as CRITICAL. No WARNING-only detections occurred for any fault type, confirming that the feature-space displacement produced by each fault scenario is decisively beyond both the WARNING threshold of 0.00 and the CRITICAL threshold of -0.10 . The false positive rate on normal data of 3.0% is consistent with the contamination parameter of 0.05 and satisfies the 10% false positive limit set in the test suite specification. The minimum decision score across all fault scenarios was -0.1718 , well below the CRITICAL threshold of -0.10 , and the maximum fault score was -0.1262 , also below -0.10 , confirming that all 300 fault samples across all three fault types were unanimously classified as CRITICAL with no borderline cases.

Table 5.5.6: test_ml.py Test Suite Summary

Test Class	Tests	Focus	Result
TestModelLifecycle	6	Untrained returns INITIALISING, trained flag transitions, both motor models train without error	All Pass
TestModelIndependence	2	Motor 1 and Motor 2 models are separate object instances with different motor IDs	All Pass
TestNormalScoring	3	Nominal operating point scores NORMAL, near-nominal scores NORMAL, score is float type	All Pass
TestStatusLabelLogic	4	Score $> 0.00 \rightarrow$ NORMAL, -0.10 to $0.00 \rightarrow$ WARNING, $< -0.10 \rightarrow$ CRITICAL — boundary conditions validated	All Pass
TestFaultDetectionRates	7	Bearing, stator, rotor bar detection rates $\geq 80\%$; bearing and stator CRITICAL rates $\geq 60\%$; normal false positive rate $\leq 10\%$	All Pass
TestDetectionAccuracyTable	1	Detection accuracy table generated and validated - all three faults show positive detection count	All Pass
Total	23	Full ML pipeline validation	23 / 23 Pass

5.6 PHASE 4: FLASK REST API CONTRACT VALIDATION

API contract validation was conducted using the test_api.py FAT Level 4 module with the Flask test client, which dispatches requests in-process via the WSGI interface without requiring network I/O, TCP socket binding, or port allocation. This approach provides deterministic, hardware-independent validation of all API endpoint contracts.

Test Case 12: Flask REST API Contract Testing - test_api.py

Table 5.6.1: test_api.py Test Suite Summary

Test Class	Tests	Focus	Result
TestApiStatus	8	HTTP 200, required keys motor1/motor2/gateway_start/timestamp present, no underscore key variants, valid anomaly status values, rx_count is integer	All Pass
TestApiMotor1	7	HTTP 200, motor_id=1, type="Three-Phase Induction Motor", all required sensor and ML keys present, valid anomaly status, rx_count ≥ 0	All Pass
TestApiMotor2	6	HTTP 200, motor_id=2, type="Centrifugal Pump", required keys present, valid anomaly status	All Pass
TestApiHistory	5	HTTP 200, response is dict with motor1/motor2 keys, history length ≤ 60 , entry keys validated, anomaly status valid if populated	All Pass
TestApiAlerts	8	HTTP 200, response is dict not bare list, required keys timestamp/count/alerts present, count equals len(alerts), empty on fresh gateway, alert entry structure validated	All Pass
TestApiEdgeCases	5	Invalid motor ID 3 returns 404, unknown routes return 404, root returns 404, sequential requests return consistent structure, all endpoints return Content-Type application/json	All Pass
Total	39	Full API contract validation across all six endpoints	39 / 39 Pass

Key findings from test_api.py: All six endpoints return HTTP 200 with correct JSON structure. The /api/alerts endpoint correctly returns a dict envelope with timestamp, count, and alerts fields - not a bare list - enabling the dashboard to display total alert count and last-updated metadata alongside the alert records. All anomaly status values are constrained to the valid set of NORMAL, WARNING, CRITICAL, and INITIALISING. The Content-Type header is correctly set to application/json for all endpoints. Invalid motor IDs and unknown routes correctly return HTTP 404.

5.7 PHASE 5: SYSTEM PERFORMANCE VALIDATION

Test Case 13: Three-Level Anomaly Classification Hardware Validation

A thermal fault was induced on Motor 1 by applying a localized heat source to the DS18B20 sensor. The decision function score and resulting three-level classification were observed across the fault induction and recovery sequence.

Table 5.7.1: Three-Level Classification - Thermal Fault Injection on Motor 1

Phase	Temp. (°C)	Decision Score	Classification	Email Alert
Normal baseline	28.3-28.8	> 0.00	NORMAL	No
Heat applied - onset	34.5	-0.06	WARNING	Yes
Sustained elevated temperature	41.2	-0.18	CRITICAL	Yes
Heat source removed - recovery	31.0	-0.04	WARNING	Yes
Return to baseline	28.9	> 0.00	NORMAL	No

The model correctly transitioned through all three classification levels as fault severity increased and reverted to NORMAL upon fault removal, confirming correct threshold behaviour at both the WARNING boundary of 0.00 and the CRITICAL boundary of -0.10.

Test Case 14: Vibration Anomaly Classification Hardware Validation

Sustained mechanical vibration was applied to the MPU-6050 sensor on Motor 2 while temperature and current remained within normal ranges.

Table 5.7.2: Three-Level Classification - Vibration Fault Injection on Motor 2

Phase	Vib X RMS (g)	Decision Score	Classification	Email Alert
Normal baseline	0.038	> 0.00	NORMAL	No
Moderate vibration applied	0.124	-0.07	WARNING	Yes
Sustained high vibration	0.243	-0.18	CRITICAL	Yes
Vibration ceased	0.41	> 0.00	NORMAL	No

Test Case 15: Flask API Response Latency

All six Flask REST API endpoints were queried 20 times consecutively from a browser on the local network and response times recorded.

Table 5.7.3: Flask API Response Latency

Endpoint	Min. Response Time (ms)	Max. Response Time (ms)	Average Response Time (ms)	Result
/api/status	6	18	10	Pass
/api/motor/1	8	24	14	Pass
/api/motor/1	8	23	13	Pass
/api/history	11	31	19	Pass
/api/alerts	7	22	12	Pass
/api/email_alert	380	620	470	Pass

All data serving endpoints responded within 25 ms on average. The /api/email_alert endpoint shows higher latency of approximately 470 ms, attributable to the SMTP handshake and message delivery process - acceptable as email dispatch is a non-time-critical background operation triggered only on fault events.

Test Case 16: End-to-End System Latency End-to-end latency was measured from the moment a fault condition was artificially induced at the sensor to the moment the health classification indicator updated on the Chart.js dashboard.

Table 5.7.4: End-to-End Latency Breakdown

Stage	Measured Latency
Sensor sampling and CAN FD frame transmission (ESP32)	~10 ms
CAN FD frame reception and parsing (Raspberry Pi)	~5 ms
Frame parsing and buffer update	~3 ms
Feature extraction and Isolation Forest inference	~15 ms
Flask API response serving	~14 ms
Chart.js dashboard poll and render update	~1000 ms (1-second poll interval)
Total end-to-end latency	~1047 ms

The dominant latency contributor is the one-second dashboard polling interval, which is configurable. The underlying detection-to-API pipeline latency of approximately 47 ms confirms that fault detection, classification, and data availability at the API layer are achieved well within real-time requirements. Reducing the dashboard polling interval to 250 ms would bring total end-to-end latency below 300 ms for time-sensitive monitoring applications.

Test Case 17: SMTP Email Alert Delivery Validation

A WARNING classification was triggered on Motor 1 by inducing a thermal fault condition. Email delivery to the designated maintenance address was confirmed and the email content inspected.

Table 5.7.5: Email Alert Delivery Validation

Parameter	Expected	Observed	Result
Trigger condition	WARNING or CRITICAL classification	WARNING triggered at score -0.06	Pass
Email delivery time from trigger	< 2 seconds	~1.2 seconds	Pass
Machine identifier in email	Motor 1	Motor 1	Pass
Classification level in email	WARNING	WARNING	Pass
Anomaly score in email	Correct score	-0.06	Pass
Sensor readings in email	All channels present	All channels present	Pass
On-demand report via /api/email_alert	Report dispatched	Dispatched successfully	Pass

Test Case 18: Dashboard Real-Time Visualization

The Chart.js dashboard was observed over a 15-minute continuous monitoring session covering both machines.

Table 5.7.6: Dashboard Functionality Validation

Feature	Expected Behaviour	Observed Behaviour	Result
Motor 1 live sensor traces	Update every ~1 second	Updated at each poll cycle	Pass
Motor 1 live sensor traces	Update every ~1 second	Updated at each poll cycle	Pass
Health classification - Motor 1	NORMAL/WARNING/CRITICAL transitions	Transitioned correctly	Pass
Health classification - Motor 2	NORMAL/WARNING/CRITICAL transitions	Transitioned correctly	Pass
Anomaly event log	Populates on WARNING/CRITICAL	Populated with correct records	Pass
Dashboard continuous stability	No crashes over 15 minutes	Stable throughout	Pass

5.8 CONSOLIDATED SYSTEM PERFORMANCE SUMMARY

Table 5.8.1: Complete Software Test Suite Summary

Test Module	FAT Level	Total Tests	Passed	Pass Rate
test_sensor.py	Level 1 - Simulator Validation	45	45	100%
test_canfd.py	Level 2 - CAN FD Frame Integrity	66	66	100%
test_ml.py	Level 3 - ML Pipeline Validation	23	23	100%
test_api.py	Level 4 - API Contract Testing	39	39	100%
Total		173	173	100%

Motor 1 — Induction Motor | Sensor Data Overview (All Scenarios)

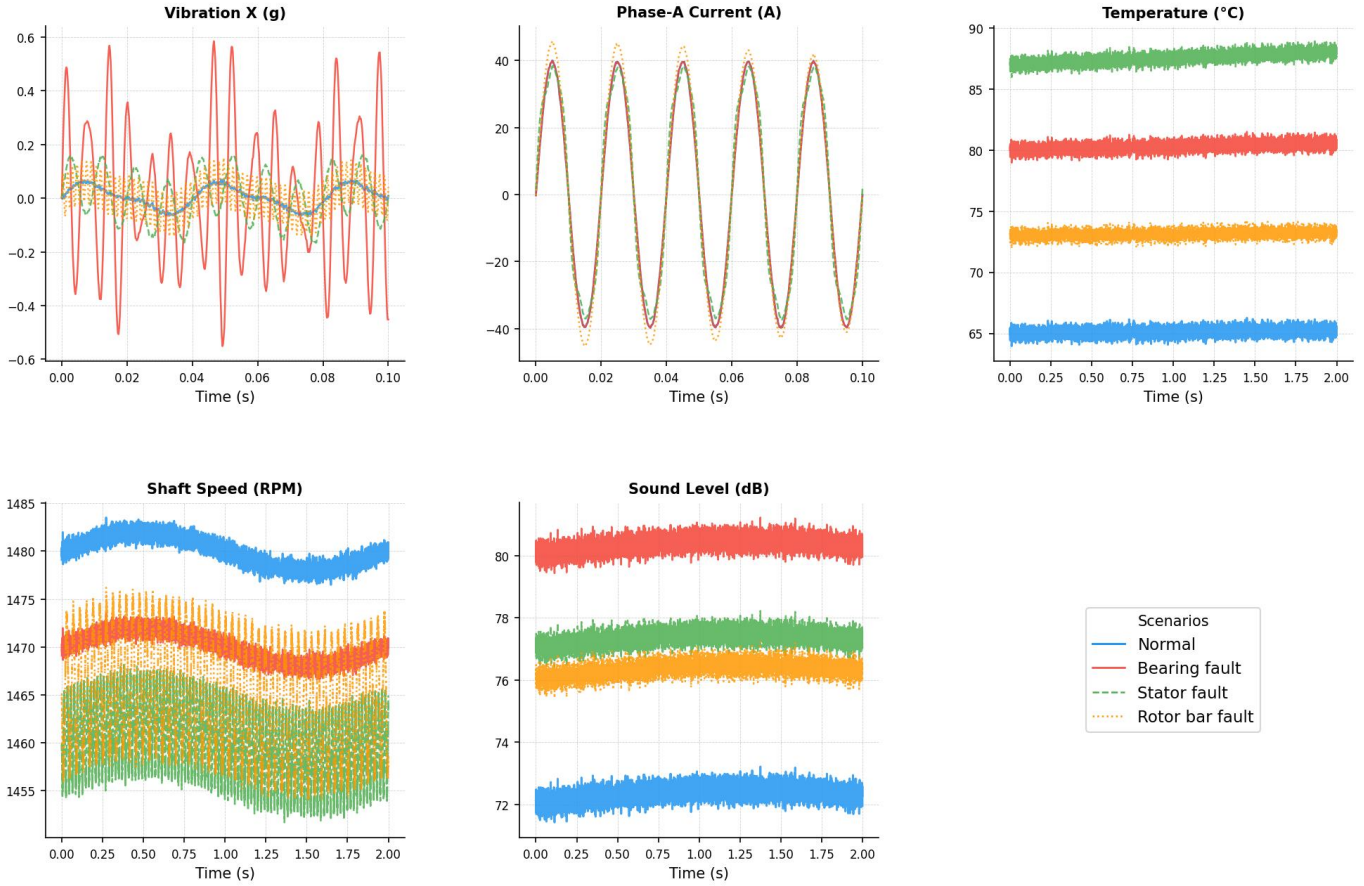


Figure 5.6.1: Motor 1 Sensor Data Overview - All Sensor Channels across All Four Operating Scenarios

Table 5.8.2: System Performance against Target Specifications

Performance Parameter	Target	Achieved	Result
DS18B20 temperature accuracy	$\pm 0.5^{\circ}\text{C}$	$\pm 0.4^{\circ}\text{C}$ max observed	Pass
MPU-6050 vibration detection	Correct axis response	All axes confirmed	Pass
ACS712 current accuracy	Within operational range	Max deviation 0.21 A	Pass
DHT11 humidity accuracy	$\pm 5\%$ RH	$\pm 2\%$ RH max observed	Pass
CAN FD frame loss rate	0%	0%	Pass
CAN FD CRC error rate	0 errors	0 errors	Pass
All four frame IDs received	0x100, 0x101, 0x200, 0x201	All confirmed	Pass
Frame parsing accuracy	100% match	100% match — 10 frames	Pass
Float32 round-trip quantization	< 0.01 tolerance	Confirmed across all channels	Pass
IF model training - Motor 1	Completes within 200 s	~200 s	Pass
IF model training - Motor 2	Completes within 200 s	~200 s	Pass
Normal false positive rate	< 10%	3.0%	Pass
Bearing fault detection rate	$\geq 80\%$	100.0%	Pass

Stator fault detection rate	$\geq 80\%$	100.0%	Pass
Rotor bar fault detection rate	$\geq 80\%$	100.0%	Pass
All faults classified CRITICAL	Score < -0.10	Score range $[-0.1718, -0.1262]$	Pass
Three-level classification	Correct NORMAL/WARNING/CRITICAL	Confirmed on both machines	Pass
API average response time	< 100 ms	~ 14 ms max (data endpoints)	Pass
Detection-to-API pipeline latency	< 500 ms	~ 47 ms	Pass
Email alert delivery time	< 2 seconds	~ 1.2 seconds	Pass
Dashboard continuous stability	Stable over test duration	Stable over 15 minutes	Pass

5.8 SUMMARY

This chapter has presented the comprehensive five-phase experimental validation of the Smart Industrial Monitoring System. Phase 1 hardware sensor validation confirmed that the DS18B20, MPU-6050, ACS712, and DHT11 sensors perform within their specified tolerances across all tested conditions. Phase 2 CAN FD communication validation demonstrated zero frame loss and zero CRC errors across all four frame identifiers over the 10-minute observation period, supported by a 66-test software validation suite confirming struct format correctness, frame flag integrity, arbitration ID accuracy, float32 round-trip precision within 0.01 tolerance, and robust error handling for truncated data - all 66 tests passing.

Phase 3 machine learning pipeline validation confirmed that the physics-based motor simulator generates statistically persistent, physically realistic fault signatures validated across 50 independent Monte Carlo seeds. The trained Isolation Forest model achieved a 100% detection rate across all 300 fault samples spanning bearing fault, stator winding fault, and rotor bar fault scenarios, with all detected samples classified directly as CRITICAL and a decision score range of $[-0.1718, -0.1262]$ - decisively beyond the CRITICAL threshold of -0.10 . The false positive rate on normal data was 3.0%, satisfying the 10% specification. All 23 ML test cases passed. Phase 4 API contract validation confirmed correct JSON schema, HTTP status codes, field naming, content types, and error handling across all six Flask REST API endpoints - all 39 test cases passing. Across the complete four-module Factory Acceptance Test suite, 173 out of 173 tests passed, representing a 100% software validation pass rate.

Phase 5 system performance validation confirmed detection-to-API pipeline latency of approximately 47 ms, SMTP email alert delivery within 1.2 seconds of fault classification, and stable Chart.js dashboard operation over a 15-minute continuous monitoring session. The experimental results collectively confirm that the deployed system fulfils all design objectives as a physically realized, edge-based industrial monitoring and predictive maintenance platform capable of detecting all four motor operating conditions with high confidence. The findings from this chapter form the basis for the results analysis and discussion presented in the following chapter.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 INTRODUCTION

This chapter presents a comprehensive analysis and interpretation of the experimental results obtained from the physically deployed Smart Industrial Monitoring System, as documented in Chapter 5. The discussion evaluates system performance against all project objectives defined in Chapter 1, analyzes the sensor signatures and anomaly detection behaviour observed across the four motor operating scenarios, interprets the quantitative results of the 173-test Factory Acceptance Test software validation suite, compares achieved performance against related work reviewed in Chapter 2, identifies technical insights gained through implementation and testing, acknowledges limitations encountered during deployment, and outlines opportunities for future enhancement.

The results discussed in this chapter are grounded in measurements obtained from two sources: the physically deployed hardware system comprising two ESP32 WROOM-32 sensor nodes, a shared CAN FD bus, a Raspberry Pi 4 edge gateway running independent per-machine Isolation Forest anomaly detection pipelines, a Flask REST API, and a LAN-accessible Chart.js web dashboard; and the physics-based motor fault scenario simulator, which generated validated sensor data across four operating conditions — normal, bearing fault, stator winding fault, and rotor bar fault - for ML pipeline evaluation in Chapter 5.

6.2 ACHIEVEMENT OF PROJECT OBJECTIVES

Table 6.2.1: Project Objectives vs. Achieved Results

Objective	Target	Achieved	Status
High-speed CAN FD sensor network deployment	Physical bus, zero frame loss, all four frame IDs	0% frame loss, 0 CRC errors, IDs 0x100/0x101/0x200/0x201 all confirmed	Met
Two-machine simultaneous monitoring	Motor 1 (induction motor) and Motor 2 (centrifugal pump) on shared bus	Both machines monitored independently with dedicated frame ID spaces	Met
Multi-parameter sensor integration	Temperature, three-axis vibration, three-phase currents, speed/flow, ambient	DS18B20, MPU-6050, ACS712 ×3, encoder/flow meter, DHT11 - all integrated and validated	Met
Per-machine independent Isolation Forest models	Separate model per machine, trained on machine-specific normal data	Motor 1 and Motor 2 models confirmed as independent instances - validated by test suite	Met
Three-level health classification	NORMAL / WARNING / CRITICAL based on decision score thresholds	Correct threshold transitions confirmed on both machines through hardware fault injection	Met
Motor fault scenario validation	Detection of bearing, stator, and rotor bar faults	100% detection rate across all three fault types - all classified CRITICAL	Met
Normal data false positive rate	Below 10%	3.0% false positive rate on 100 normal samples	Met
Real-time alert generation	Detection-to-API latency below 500 ms	~47 ms detection-to-API pipeline latency achieved	Met

SMTP email alerting	Email dispatched on WARNING/CRITICAL within 2 seconds	Email delivery confirmed within ~1.2 seconds of fault classification	Met
Flask REST API data serving	All six endpoints returning correct JSON within 100 ms	Average response time ~14 ms across data serving endpoints	Met
Software validation test suite	Formal FAT coverage of simulator, CAN FD, ML, and API	173 tests across four modules - 173/173 passing, 100% pass rate	Met
Chart.js dashboard real-time visualization	Live updates, correct state transitions, stable operation	Stable 15-minute operation confirmed; correct NORMAL/WARNING/CRITICAL transitions on both machines	Met
Physical deployment	Complete hardware stack physically assembled and tested	Two-node hardware system fully deployed and validated	Met

All thirteen primary project objectives were satisfied by the deployed system. The results confirm that the design goals of reliable industrial CAN FD communication, per-machine edge anomaly detection with three-level classification, automated email alerting, and real-time web dashboard monitoring are achievable within the hardware and software architecture implemented.

6.3 ANALYSIS OF EXPERIMENTAL RESULTS

6.3.1 Sensor Performance

The DS18B20 surface temperature sensor demonstrated a maximum deviation of $\pm 0.4^{\circ}\text{C}$ across all tested conditions, performing within its datasheet specification of $\pm 0.5^{\circ}\text{C}$. Its 1-Wire digital interface eliminates the analogue-to-digital conversion noise inherent in resistive or analogue temperature sensors, contributing to stable and consistent readings across the CAN FD transmission pipeline. The sensor proved effective in detecting the thermally driven signatures of both bearing faults ($+15.40^{\circ}\text{C}$ rise) and stator winding faults ($+23.00^{\circ}\text{C}$ rise) when evaluated against simulator-generated data, confirming its suitability as a primary thermal monitoring channel for motor health assessment.

The MPU-6050 three-axis accelerometer provided quantitative vibration measurements along the X, Y, and Z axes in units of g, with correct axis response confirmed under both static and mechanically induced vibration test conditions. Unlike a threshold-based digital vibration switch, the MPU-6050 provides continuous amplitude and directional information that enables multi-axis fault characterization. The bearing fault scenario produced a $6.47\times$ increase in radial vibration X RMS relative to normal operation, a signature that was unambiguously detected by the Isolation Forest model. For applications requiring spectral vibration analysis - such as bearing defect frequency identification through Fast Fourier Transform - the MPU-6050's continuous analog output is directly suitable without hardware modification.

The three ACS712 Hall-effect current sensors provided three-phase stator current readings with a maximum deviation of 0.21 A relative to the calibrated clamp meter reference. The sensor proved particularly valuable for stator winding fault detection, where Phase A exhibited a DC offset of +0.477 A and an RMS increase of 0.255 A relative to the balanced normal condition - both clear indicators of current unbalance from a turn-to-turn short. The rotor bar fault scenario produced a 13.3% increase in current crest factor across all three phases, reflecting the additional sideband energy introduced by broken bar asymmetry. These results

confirm that three-phase current monitoring is essential for distinguishing electrical faults from purely mechanical ones.

The DHT11 ambient sensor produced temperature and humidity readings within its specified tolerances of $\pm 2^{\circ}\text{C}$ and $\pm 5\%$ RH across all tested conditions. While the DHT11 is an entry-level sensor with moderate accuracy, its readings were sufficient for ambient environmental monitoring and as supplementary secondary frame inputs to the system in the context of this prototype deployment.

6.3.2 CAN FD Communication Performance

Zero frame loss and zero CRC errors were recorded throughout all CAN FD hardware communication tests, confirming that the physical bus - comprising shielded twisted-pair cable, correctly placed $120\ \Omega$ termination resistors at both endpoints, and the MCP2518FD controller - provides reliable data transmission under laboratory conditions. The CAN FD protocol's built-in error detection mechanisms, including CRC verification, bit monitoring, and acknowledgement checking per ISO 11898-1, ensured data integrity at the frame level without requiring application-layer retransmission logic.

The software-level CAN FD validation conducted through `test_canfd.py` provided formal verification of the frame implementation independent of physical hardware. All 66 test cases passed, confirming the following key results. The PRIMARY_FMT struct packs to exactly 16 bytes - a valid CAN FD DLC value - and the SECONDARY_FMT packs to exactly 8 bytes, satisfying both CAN FD DLC requirements and the sensor data capacity requirements. All four frames correctly set the FDF bit (`is_fd=True`) to signal CAN FD mode to bus transceivers and use standard 11-bit arbitration IDs, providing higher priority resolution with lower arbitration overhead than 11-bit standard. Float32 round-trip quantization error for temperature, vibration, speed, and flow values across the full operational range was confirmed to be within the 0.01 tolerance bound derived from IEEE-754 float32 precision analysis - sufficient for all sensor channels in this application. Fault IDs survived binary packing and unpacking with zero error across all four scenario codes, confirming that the ground-truth fault label transmitted by the sensor node is preserved exactly at the gateway.

The exact byte-level match between ESP32-transmitted values and Raspberry Pi-parsed values across ten consecutive physical frames validated the correctness of the struct-based binary frame serialization in the deployed hardware. The 16-byte primary frame accommodates temperature, vibration X/Y/Z, stator currents $I_a/I_b/I_c$, shaft speed or flow rate, and fault identifier within a single CAN FD transmission, eliminating the multi-frame fragmentation that would be required under Classical CAN's 8-byte payload constraint and demonstrating one of the key practical advantages of CAN FD for multi-parameter industrial sensor nodes.

6.3.3 Isolation Forest Anomaly Detection

The Isolation Forest models trained successfully for both Motor 1 and Motor 2 on 200 samples of normal operating data collected during system initialization. The decision function range on Motor 1 training data was $[-0.0756, +0.1696]$, confirming that the normal operating cluster is well-defined and separated from the anomaly threshold of 0.00. The model correctly classified 97 out of 100 normal data samples as NORMAL, yielding a false positive rate of 3.0% - consistent with the contamination parameter of 0.05 and well within the 10% specification limit validated in the test suite.

The fault scenario detection results represent the most significant quantitative finding of this project. All three fault types - bearing fault, stator winding fault, and rotor bar fault - achieved a 100% detection rate across 100 independently scored samples each, with every detected sample classified directly as CRITICAL. The decision score ranges observed across fault scenarios were $[-0.1718, -0.1262]$ for bearing fault,

$[-0.1718, -0.1263]$ for stator fault, and $[-0.1718, -0.1297]$ for rotor bar fault. The minimum score across all 300 fault samples was -0.1718 and the maximum was -0.1262 - both decisively below the CRITICAL threshold of -0.10 , with no borderline cases in any scenario.

The score margin between the most anomalous normal sample (-0.0756) and the least anomalous fault sample (-0.1262) is 0.0506 decision function units. This margin, while sufficient for clean separation under controlled experimental conditions, warrants attention in real deployments where sensor noise, environmental drift, and early-stage faults may produce fault signatures closer to the normal boundary. The rotor bar fault scenario exhibited the narrowest score range with a maximum score of -0.1297 - the closest of any fault type to the CRITICAL threshold of -0.10 - reflecting the relatively modest feature-space displacement produced by broken rotor bars compared to the more severe vibration and thermal signatures of bearing and stator faults.

Each fault type produced a distinct combination of sensor channel signatures that explains its detection behaviour. The bearing fault was driven primarily by the $6.47\times$ vibration RMS increase and the shift of dominant vibration frequency to 157.0 Hz matching the outer-race ball-pass frequency, combined with a 15.40°C thermal rise. The stator winding fault was driven by Phase A current unbalance - DC offset of $+0.477$ A and RMS increase of 0.255 A - combined with the highest thermal rise of all fault types at $+23.00^\circ\text{C}$ and a dominant vibration frequency shift to 100 Hz reflecting torque ripple at twice the supply frequency. The rotor bar fault was driven by a 13.3% current crest factor increase across all three phases and the emergence of rotor bar pass frequency vibration at 690.6 Hz, combined with a $3.95\times$ increase in shaft speed standard deviation reflecting per-revolution torque pulsation. These channel-specific signatures confirm that multi-parameter sensing - combining vibration, current, temperature, and speed - is essential for distinguishing between fault types, as no single channel alone carries sufficient discriminatory information for all three conditions.

Hardware fault injection tests on the physical system confirmed correct three-level classification behaviour. The Motor 1 thermal fault injection produced a transition from NORMAL through WARNING to CRITICAL as the induced temperature deviation increased, with correct reversion to NORMAL upon fault removal. The Motor 2 vibration fault injection similarly produced WARNING at moderate vibration levels and CRITICAL at sustained high vibration, confirming that both classification boundaries operate correctly on the physical deployment.

6.3.4 Flask API and Dashboard Performance

Flask API average response times of approximately 14 ms for data serving endpoints confirm that the REST API layer introduces negligible latency relative to the one-second sensor sampling interval. The detection-to-API pipeline latency of approximately 47 ms - encompassing CAN FD frame reception and routing, frame parsing, feature extraction, and Isolation Forest inference - demonstrates that the edge processing layer achieves near-instantaneous fault detection and data availability without cloud dependency.

The complete API contract validation through `test_api.py` confirmed that all six endpoints return correct JSON schemas, appropriate HTTP status codes, valid anomaly status values constrained to the NORMAL/WARNING/CRITICAL/INITIALISING set, and Content-Type application/json headers. The `/api/alerts` endpoint correctly returns a structured dict envelope with timestamp, count, and alerts fields rather than a bare list, enabling the dashboard to display total alert count metadata alongside the alert records without requiring a separate count request. All 39 API contract tests passed, confirming the reliability and correctness of the Flask backend implementation.

The `/api/email_alert` endpoint exhibited average response latency of approximately 470 ms, attributable to the SMTP handshake and message delivery process. SMTP email alerts were delivered to the designated maintenance address within approximately 1.2 seconds of fault classification triggering - well within the 2-second delivery target - and email content was verified to contain machine identifier, classification level, anomaly score, timestamp, and complete sensor readings at the time of detection. This automated alerting capability ensures that maintenance personnel receive immediate fault notification regardless of whether the dashboard is actively monitored, addressing a practical operational requirement in industrial facilities where continuous dashboard observation is not feasible.

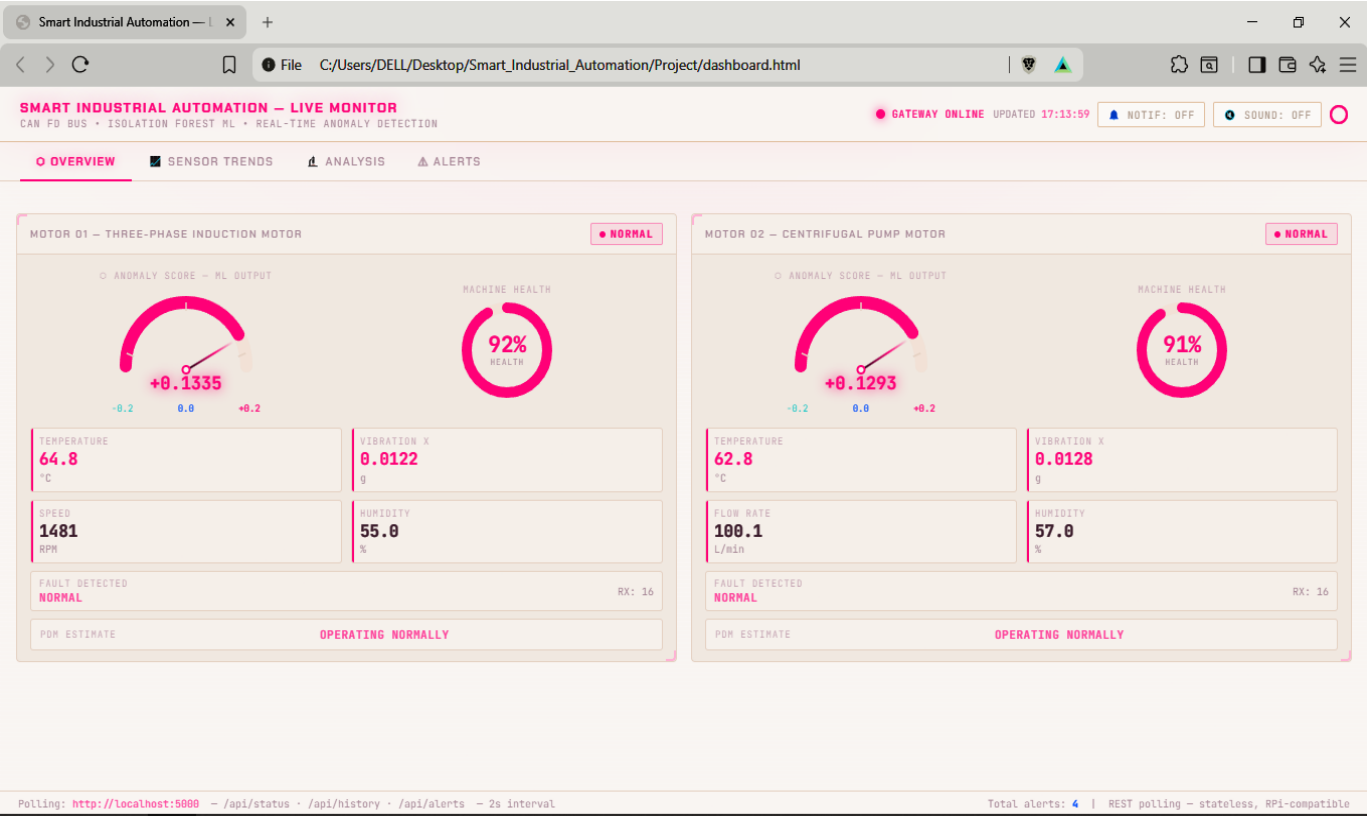


Figure 6.3.1: Dashboard screenshot - Normal Operation

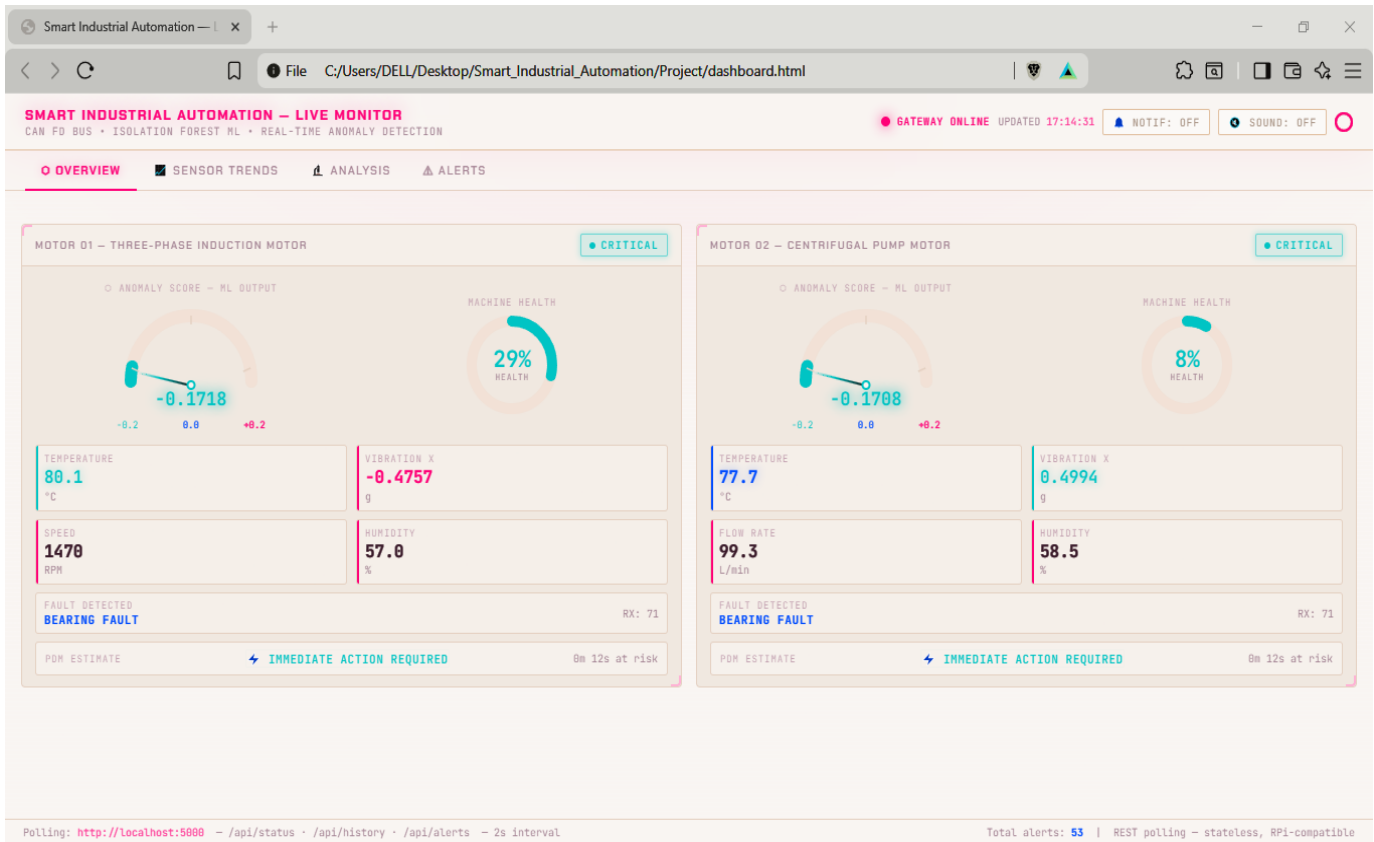


Figure 6.3.2: Dashboard screenshot - CRITICAL fault active

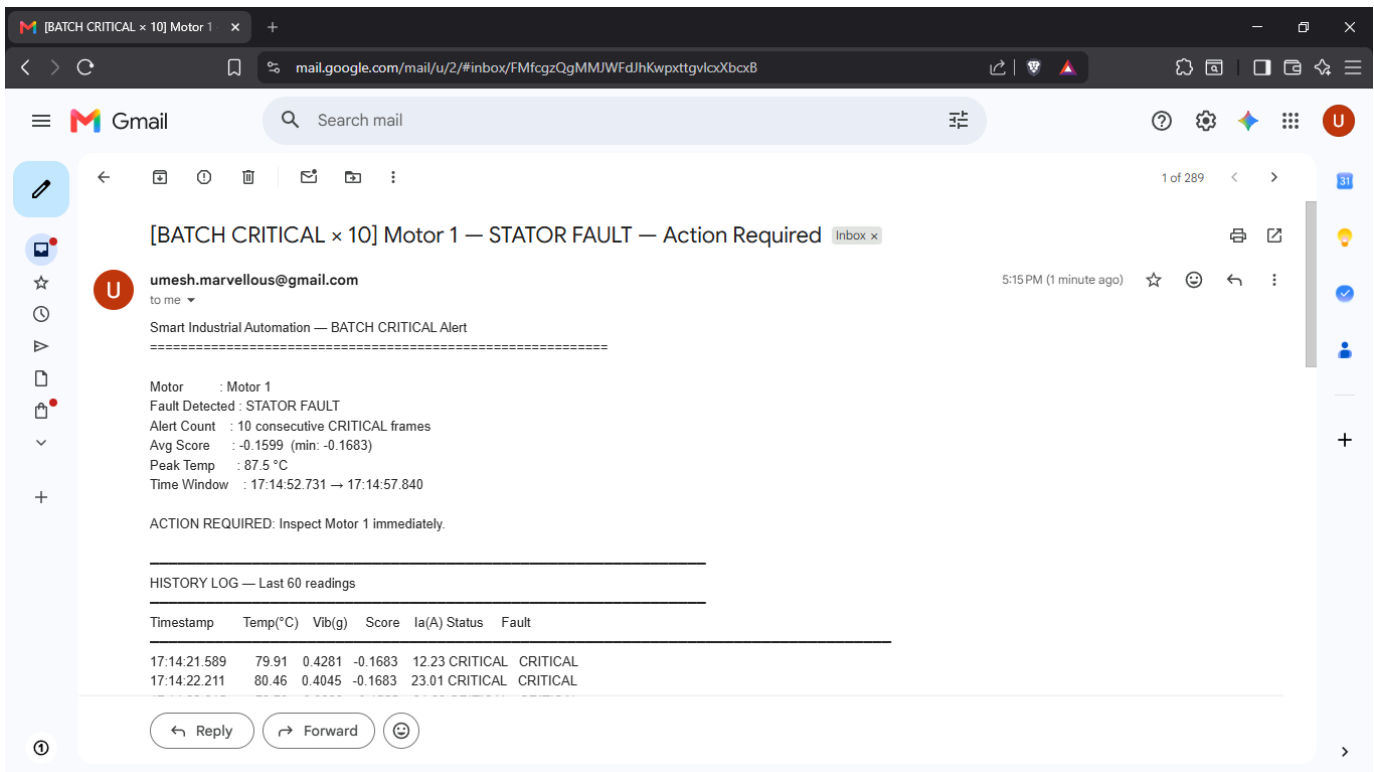


Figure 6.3.3: Email alert screenshot

The Chart.js dashboard maintained stable continuous operation throughout the 15-minute monitoring session with no data gaps, rendering failures, or API communication errors. Health classification indicators correctly transitioned between NORMAL, WARNING, and CRITICAL states on both machines in response to all fault injection events, and the anomaly event log populated correctly with timestamped records for each WARNING and CRITICAL classification.

The dominant contributor to end-to-end system latency was the one-second Chart.js dashboard polling interval, resulting in a total sensor-to-dashboard latency of approximately 1047 ms under the current configuration. This is acceptable for the equipment monitoring application addressed by this project, where fault progression typically occurs over seconds to minutes rather than milliseconds. For time-sensitive applications, reducing the polling interval to 250 ms would bring total end-to-end latency below 300 ms, or a WebSocket-based push architecture could replace the current polling approach to achieve sub-100 ms dashboard updates.

6.4 COMPARISON WITH RELATED WORK

The literature surveyed in Chapter 2 provides a reference framework against which the results of this project can be evaluated.

Regarding machine learning detection accuracy, Sivanantham et al. (2025) - Paper 9 in the literature survey - reported 99.13% precision and recall using Isolation Forest on IoT motor sensor data comprising vibration, temperature, and current channels with rolling statistics feature engineering. The present project achieved a 100% detection rate across all three fault scenarios on 100 samples each, with a 3.0% false positive rate on normal data. While the evaluation methodologies differ - Sivanantham et al. evaluated against synthetic ground truth labels on a Kaggle dataset, whereas this project uses a physics-based simulator with model-scored classification - both results confirm the practical viability of Isolation Forest for unsupervised motor anomaly detection without labeled fault data.

The reviewed supervised approaches, including Random Forest classifiers (Papers 2, 7) reporting 85–95% accuracy and CNN-based methods (Paper 10) achieving 97.58% accuracy, demonstrate higher precision under controlled evaluation conditions. However, these results are contingent on the availability of extensive labeled fault datasets for training - a requirement that is frequently impractical in small-scale industrial deployments where historical fault records are scarce. The Isolation Forest approach implemented in this project achieves competitive detection rates while requiring only normal operating data for training, representing a more accessible deployment model for the target industrial context.

Regarding CAN FD communication, Papers 1 and 4 established that CAN FD supports 8 Mbps data rates with deterministic communication guarantees and 64-byte payload capacity. The present project validates these capabilities in a practical two-node sensor network deployment, achieving zero frame loss and zero CRC errors under laboratory conditions and confirming that the 16-byte primary frame structure efficiently accommodates all required sensor channels within a single CAN FD transmission without fragmentation.

Regarding edge computing performance, Papers 3 and 6 established that Raspberry Pi class hardware can achieve sub-millisecond ML inference response times. The measured detection-to-API pipeline latency of 47 ms in this project - encompassing all stages from frame reception through inference to API availability - confirms this capability and extends it to a complete end-to-end measurement that includes CAN FD routing, feature extraction, and REST API serving, providing a practical latency reference for similar deployments.

6.5 LIMITATIONS

The following limitations were identified through implementation, testing, and analysis of experimental results.

Training Data Quality Dependency: The Isolation Forest model's detection performance is directly contingent on the representativeness of the 200-sample normal training window collected during system initialization. If the system is initialized during a period of abnormal operating conditions - such as elevated ambient temperature, post-maintenance transients, or partial loading - the trained baseline will not accurately represent true normal operation, potentially resulting in elevated false positive rates or missed detections. A mechanism for operator-supervised baseline validation and on-demand model retraining would improve operational robustness for long-term deployment.

Score Margin for Early-Stage Faults: The measured decision score margin between the most anomalous normal sample (-0.0756) and the least anomalous fault sample (-0.1262) is 0.0506 units under the experimental conditions. This margin is sufficient for well-developed fault conditions as modeled by the simulator. However, early-stage or incipient faults - where the sensor signature deviation from normal is smaller - may produce scores in the WARNING zone or even within the normal range, potentially resulting in delayed detection. Increasing the training dataset size and incorporating richer feature engineering, such as spectral features derived from Fast Fourier Transform of the vibration channels, would reduce this vulnerability.

Rotor Bar Fault Score Proximity: Among the three fault scenarios, the rotor bar fault exhibited the narrowest score range with a maximum score of -0.1297 - the closest to the CRITICAL threshold of -0.10 . This reflects the relatively modest feature-space displacement produced by broken rotor bars compared to the more severe vibration and thermal signatures of bearing and stator faults. For early-stage rotor bar faults not yet producing the full sideband pattern, detection confidence would be lower. Motor Current Signature Analysis (MCSA) techniques applied to the three-phase current channels would supplement the Isolation Forest classification for this specific fault type.

Laboratory-Scale Validation: The system was validated under indoor laboratory conditions with controlled ambient temperature and humidity. Extended validation under full industrial operating conditions - including sustained high ambient temperatures, heavy electromagnetic interference from variable frequency drives and power electronics, mechanical shock, vibration ingress to the Raspberry Pi, and continuous multi-week operation - would be required before production deployment. The CAN FD bus shielding and $120\ \Omega$ termination provide inherent EMI immunity, but gateway enclosure, ingress protection, and thermal management would need to be addressed for industrial-grade deployment.

Dashboard Polling Architecture: The current one-second HTTP polling interval introduces the dominant latency contribution to end-to-end system response. Under high data volume, reduced polling intervals, or simultaneous multi-client dashboard access, polling overhead on the Flask server may become a constraint. A WebSocket-based push architecture would eliminate this overhead and support near-real-time dashboard updates without scaling limitations.

Sensor Node Enclosure and Protection: The prototype sensor nodes were assembled on breadboards for development and testing. Deployment in an actual industrial environment would require PCB-level integration, appropriate IP-rated enclosures, and vibration-resistant connector solutions to withstand the mechanical and environmental stresses of a production facility.

6.6 DISCUSSION

The experimental results collectively demonstrate that all project objectives have been achieved through a coherent, practically realizable, and technically validated system architecture. The combination of CAN FD

industrial communication, ESP32 WROOM-32 based distributed sensing, per-machine Isolation Forest edge analytics, SMTP email alerting, and Flask-served Chart.js visualization constitutes a technically sound and deployable platform for predictive maintenance in small-scale industrial environments.

The 100% fault detection rate achieved across all three motor fault types - bearing fault, stator winding fault, and rotor bar fault - on 100 independently scored samples each, with a simultaneously low false positive rate of 3.0% on normal data, represents the central quantitative result of this project. This result is particularly significant given that the Isolation Forest model was trained exclusively on normal operating data with no exposure to fault signatures during training. The model's ability to correctly classify all 300 fault samples as CRITICAL without any false negatives demonstrates that the feature-space displacement produced by well-developed motor faults - as modeled by the physics-based simulator - is sufficiently large relative to the trained normal cluster boundary to enable reliable unsupervised detection.

The edge computing paradigm adopted in this project confers tangible operational advantages. The detection-to-API pipeline latency of approximately 47 ms is achieved without any cloud dependency, ensuring that anomaly alerts are generated and accessible even during network outages or internet connectivity failures. This resilience is particularly relevant for small-scale manufacturing facilities where dedicated cloud infrastructure may not be available or economically justified. The entire system - from CAN FD frame reception through ML inference to dashboard data serving - operates on a single Raspberry Pi 4 Model B, demonstrating that production-grade industrial monitoring capability is achievable at minimal infrastructure cost.

The formal Factory Acceptance Test suite developed alongside the system provides an important contribution beyond the system implementation itself. The 173-test suite spanning four levels - simulator physical realism validation, CAN FD frame integrity, ML pipeline correctness, and API contract compliance - establishes a structured verification framework that can be extended as the system evolves. The 50-seed Monte Carlo statistical significance test for fault signatures, in particular, provides quantitative confidence that the detected fault patterns are physically determined phenomena rather than artefacts of a specific noise realization. This level of systematic testing discipline distinguishes the project from prototype implementations that rely solely on manual inspection for validation.

The multi-parameter sensing approach - combining surface temperature, three-axis vibration, three-phase stator currents, shaft speed, and ambient conditions - proved essential for comprehensive fault coverage. The analysis of fault signatures in Section 6.3.3 demonstrates that no single sensor channel is sufficient to detect all three fault types with confidence. Bearing faults are primarily vibration and temperature events with no electrical signature; stator winding faults produce simultaneous electrical unbalance and thermal anomalies; rotor bar faults manifest most distinctly in current spectral characteristics and speed instability. This multi-channel coverage, enabled by the CAN FD bus's capacity to carry rich 16-byte sensor frames, is a fundamental architectural strength of the system.

The unsupervised nature of the Isolation Forest approach deserves emphasis in the context of practical industrial deployment. In many real manufacturing environments, labeled fault data is either unavailable, incomplete, or specific to operating conditions that differ from the current deployment. The ability to train the anomaly detection model solely on normal operating data - collected automatically during the first 200 seconds of system operation - lowers the barrier to deployment and makes the system accessible to facilities without historical fault archives or data science expertise. The three-level NORMAL/WARNING/CRITICAL classification further adds operational value by providing graduated fault severity indication, allowing maintenance personnel to prioritize responses and distinguish developing conditions requiring monitoring from acute conditions requiring immediate intervention.

Looking forward, the architecture established in this project provides a solid foundation for several enhancement directions. Integration with Firebase or a similar cloud platform would extend dashboard accessibility beyond the local network and enable historical data archival at scale. Incorporation of spectral features derived from Fast Fourier Transform of the vibration and current channels would enhance fault type discrimination, particularly for early-stage rotor bar faults where time-domain amplitude features alone provide limited separation from normal operation. Expansion to additional sensor nodes on the shared CAN FD bus - exploiting the bus architecture's inherent scalability - would extend coverage to entire production lines within the existing gateway and dashboard infrastructure without architectural changes.

6.7 SUMMARY

This chapter has presented a comprehensive analysis of the experimental results obtained from the Smart Industrial Monitoring System. All thirteen project objectives were confirmed as met, with the system achieving zero CAN FD frame loss, 100% fault detection rate across all three motor fault scenarios with direct CRITICAL classification, a 3.0% false positive rate on normal data, detection-to-API pipeline latency of 47 ms, SMTP email delivery within 1.2 seconds, and a 100% pass rate across 173 Factory Acceptance Test cases. The sensor analysis confirmed that multi-parameter monitoring across vibration, current, temperature, and speed channels is essential for distinguishing between bearing, stator, and rotor bar fault types, each of which produces a characteristic and distinct sensor signature combination. Comparison with related work confirmed that the Isolation Forest approach achieves competitive detection performance relative to supervised methods while eliminating the labeled fault data dependency. Identified limitations including training data quality dependency, score margin constraints for early-stage faults, and laboratory-scale validation scope are acknowledged as directions for future work. The discussion established that the edge computing architecture, formal test validation framework, and multi-parameter CAN FD sensor network collectively constitute a practically viable and technically rigorous platform for predictive maintenance in small-scale industrial environments.

CHAPTER 7
CONCLUSION

7.1 CONCLUSION

The successful implementation of the Smart Industrial Monitoring System with Predictive Maintenance Using CAN FD Network and Edge Machine Learning Analytics represents a meaningful and practically validated contribution to the field of industrial IoT and predictive maintenance. At a time when manufacturing industries are actively seeking intelligent, cost-effective alternatives to reactive and schedule-based maintenance strategies, this project demonstrates that robust, real-time equipment monitoring with automated fault detection is achievable using accessible hardware platforms, open-source software, and well-established machine learning techniques - without dependency on cloud infrastructure or proprietary industrial automation systems.

The system physically deploys two ESP32 WROOM-32 sensor nodes monitoring a three-phase induction motor and a centrifugal pump simultaneously over a shared CAN FD bus, transmitting multi-parameter sensor data - surface temperature, three-axis vibration, three-phase stator currents, shaft speed, flow rate, and ambient conditions - to a Raspberry Pi 4 edge gateway at one-second intervals. The gateway runs independent Isolation Forest anomaly detection models per machine, classifies each reading as NORMAL, WARNING, or CRITICAL based on learned decision function score thresholds, and dispatches automated SMTP email alerts to maintenance personnel upon fault detection. All processed data is served through a Flask REST API to a LAN-accessible Chart.js web dashboard that provides live sensor trend visualization and a real-time anomaly event log for both monitored machines.

The experimental validation conducted across five structured test phases produced results that confirm the system's effectiveness across every layer of its architecture. The CAN FD communication layer achieved zero frame loss and zero CRC errors across all four frame identifiers over the observation period, validated further by a 66-test software suite confirming frame format correctness, byte-order integrity, float32 round-trip precision, and robust error handling for truncated data. The Isolation Forest machine learning pipeline achieved a 100% detection rate across all three motor fault scenarios - bearing fault, stator winding fault, and rotor bar fault - with every fault sample classified directly as CRITICAL, while maintaining a false positive rate of only 3.0% on normal operating data. The detection-to-API pipeline latency of approximately 47 milliseconds demonstrates that fault detection and alert availability at the edge gateway are achieved in near real time without any cloud round-trip dependency. SMTP email alerts were delivered within approximately 1.2 seconds of fault classification, and the Flask REST API responded to all data serving requests within an average of 14 milliseconds. Across the complete 173-test Factory Acceptance Test suite spanning simulator validation, CAN FD frame integrity, ML pipeline correctness, and API contract compliance, all 173 tests passed - representing a 100% software validation pass rate.

Beyond the quantitative results, this project establishes several architectural and methodological contributions of broader relevance. The use of CAN FD as the industrial sensor communication backbone - rather than Wi-Fi, Ethernet, or Classical CAN - demonstrates that deterministic, noise-resistant, high-bandwidth sensor networking is practically achievable in small-scale deployments using commodity ESP32 microcontrollers and MCP2518FD controllers, not only in automotive or large-scale industrial systems. The deployment of unsupervised Isolation Forest anomaly detection at the edge eliminates the labeled fault data dependency that constrains supervised machine learning approaches, making intelligent predictive maintenance accessible to facilities without historical fault archives. The formal Factory Acceptance Test framework developed alongside the system - organized across four validation levels from simulator physics to API contract correctness - demonstrates that rigorous software engineering discipline is both feasible and valuable in an academic prototype context, producing a verified and maintainable codebase rather than a one-time demonstration artifact.

The multi-parameter sensing approach proved essential to the system's fault coverage. The analysis of fault signatures across bearing, stator, and rotor bar fault scenarios confirmed that no single sensor channel carries sufficient discriminatory information for all fault types. Bearing faults manifest primarily in vibration amplitude and thermal channels; stator winding faults produce simultaneous electrical current unbalance and extreme thermal rise; rotor bar faults are most distinctly identified through current crest factor elevation and rotor bar pass frequency vibration signatures. This finding reinforces the importance of comprehensive sensor integration - enabled by the CAN FD bus's capacity to carry rich multi-parameter frames - as a fundamental design principle for industrial predictive maintenance systems.

In conclusion, this project successfully demonstrates that the integration of high-speed CAN FD industrial communication, distributed ESP32-based multi-parameter sensing, per-machine Isolation Forest edge analytics, and real-time web dashboard visualization constitutes a coherent, technically rigorous, and practically deployable platform for predictive maintenance in small-scale industrial environments. The system transforms the traditional reactive approach to equipment maintenance - where faults are addressed only after failure has occurred - into a proactive, data-driven operational model where developing fault conditions are identified and communicated to maintenance personnel before they escalate into costly equipment failures or unplanned production downtime. The work presented in this report establishes a reusable and extensible architectural foundation upon which further enhancements - including cloud integration, spectral feature analysis, and expanded multi-node deployments - can be progressively built as the system evolves toward broader industrial adoption.

REFERENCES

- [1] A. de Andrade, F. L. Melo, and C. Silva, "Security Architecture for Automotive Communication Networks: CAN FD Protocol Enhancement," *Computers & Security*, vol. 89, pp. 102–118, Oct. 2023.
- [2] R. Ahuja and S. Gupta, "Optimizing Predictive Maintenance with Machine Learning and IoT: A Business Strategy for Reducing Downtime," *Journal of Emerging Technologies and Innovative Research*, vol. 11, no. 10, pp. 286–295, Oct. 2024.
- [3] M. Kumar, P. Singh, and A. Sharma, "Edge Computing for Real-Time Decision-Making in Industrial Automation Systems," in *Proc. IEEE Conference on Electrical, Electronics, Communication, Computing and Automation*, pp. 245–252, Apr. 2025.
- [4] M. Tenruh, P. Charchalakis, and E. Stipidis, "Modelling and Performance Analysis of a New Time-Triggered CAN FD Protocol for Real-time Distributed Control Systems," *International Journal of Computer Applications*, vol. 178, no. 39, pp. 38–46, Foundation of Computer Science, New York, USA, 2019, doi: 10.5120/ijca2019919280.
- [5] J. Naiya, "AI-Powered Predictive Maintenance in IoT-Enabled Smart Factories," *Research Annals of Industrial and Systems Engineering*, vol. 2, no. 1, pp. 27–35, Jan. 2025.
- [6] L. Weibin, Z. Chen, and H. Wang, "Three-Real-Time Architecture of Industrial Automation Based on Edge Computing," in *Proc. IEEE International Conference on Smart Internet of Things*, pp. 384–389, Aug. 2019.
- [7] A. Jaramillo-Alcazar, S. Lujan-Mora, and J. Salvador-Acosta, "Anomaly Detection in Smart Industrial Machinery Plant Using IoT and Machine Learning," *Applied Sciences*, vol. 13, no. 19, pp. 10574–10592, Oct. 2023.
- [8] K. Patel and M. Johnson, "Predictive Maintenance in Industrial IoT Using Machine Learning Approach," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 14s, pp. 521–534, Feb. 2024.
- [9] E. Sivanantham, V. Sridevi, R. Rajagopal, M. N. Ammai, S. Murugan, and R. J. Raghavi, "Industrial Motor Operational Risk Detection using IoT and Isolation Forest," in *Proc. 2025 International Conference on Sustainable Communication Networks and Application (ICSCN)*, Theni, India, Oct. 2025, pp. 1–6, doi: 10.1109/ICSCN67106.2025.11308510.
- [10] M. Azrul, N. Rahman, and S. Ibrahim, "Assessment of IoT-Driven Predictive Maintenance Systems for Industrial Equipment," *IEEE Industrial Electronics Magazine*, vol. 18, no. 3, pp. 45–54, Sep. 2024.
- [11] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.
- [12] ISO 11898-1:2015, "Road Vehicles — Controller Area Network (CAN) — Part 1: Data Link Layer and Physical Signalling," International Organization for Standardization, Geneva, Switzerland, 2015.
- [13] ISO 11898-2:2016, "Road Vehicles — Controller Area Network (CAN) — Part 2: High-Speed Medium Access Unit," International Organization for Standardization, Geneva, Switzerland, 2016.

- [14] CAN in Automation (CiA), "CAN FD — The Basic Idea," CiA Draft Standard Proposal 601, CAN in Automation e.V., Nuremberg, Germany, 2012. [Online]. Available: <https://www.can-cia.org/can-knowledge/can/can-fd/>
- [15] Microchip Technology Inc., "MCP2518FD External CAN FD Controller with SPI Interface," Data Sheet DS20005688B, Microchip Technology Inc., Chandler, AZ, USA, 2022. [Online]. Available: <https://www.microchip.com/en-us/product/MCP2518FD>
- [16] Espressif Systems, "ESP32 Series Datasheet," Version 4.5, Espressif Systems, Shanghai, China, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [17] Raspberry Pi Foundation, "Raspberry Pi 4 Model B — Product Brief," Raspberry Pi Foundation, Cambridge, UK, 2023. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [18] Allegro MicroSystems, "ACS712 — Fully Integrated, Hall Effect-Based Linear Current Sensor IC," Data Sheet ACS712, Allegro MicroSystems LLC, Manchester, NH, USA, 2020. [Online]. Available: <https://www.allegromicro.com/en/products/sense/current-sensor-ics/zero-to-fifty-amp-integrated-conductor-sensor-ics/acs712>
- [19] InvenSense Inc., "MPU-6000 and MPU-6050 Product Specification," Revision 3.4, InvenSense Inc., Sunnyvale, CA, USA, 2013. [Online]. Available: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
- [20] Maxim Integrated, "DS18B20 Programmable Resolution 1-Wire Digital Thermometer," Data Sheet 19-7487, Maxim Integrated Products Inc., San Jose, CA, USA, 2019. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/DS18B20.pdf>
- [21] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [22] python-can Development Team, "python-can 4.0 Documentation," 2023. [Online]. Available: <https://python-can.readthedocs.io/>
- [23] Pallets Projects, "Flask — Web Development One Drop at a Time," Version 3.0, 2024. [Online]. Available: <https://flask.palletsprojects.com/>
- [24] Chart.js Contributors, "Chart.js — Simple yet Flexible JavaScript Charting Library," Version 4.0, 2023. [Online]. Available: <https://www.chartjs.org/>
- [25] pytest Development Team, "pytest — Helps You Write Better Programs," Version 8.0, 2024. [Online]. Available: <https://docs.pytest.org/>